GOLDEN RASTREAMENTO

Instalação completa do sistema

Site, captação de leads, painel (CRM, CMS e ERP), loja virtual e portal do cliente — do primeiro `git clone` ao checklist de saída.

Versão **2.36.0** Data **26/10/2026** Requisito **PHP 8.3 + MySQL**

Responsável **Marcelo · KDKHOST** Contato **contato@kdkhost.com.br**

1. O que a instalação faz

1. Confere PHP, extensões e Composer antes de mexer em qualquer coisa.
2. Cria o arquivo `.env` de produção (chave da aplicação, endereço, banco, fuso horário, filas e cache) com permissão `600`.
1. Instala as dependências do projeto sem as bibliotecas de desenvolvimento.
2. Prepara o banco: se ele já tiver tabelas de outro sistema, tira uma cópia de segurança e recria as tabelas do Golden a partir das migrations.
1. Grava os dados de demonstração (catálogo, páginas do site, leads de exemplo) — ou só o catálogo base, com `--sem-demo`.
1. Define o login do painel.
2. Cria o link `public/storage` para as imagens, vídeos e documentos enviados.
3. Escreve um `.htaccess` na raiz que bloqueia código-fonte, `.env` e logs, faz todo pedido entrar pela pasta `public` e, com `--php-web=`, escolhe o PHP que serve a web.
1. Liga os caches de configuração, rotas e views.

2. Abre o próprio `--url` no final e avisa se o PHP da web for outro.

Nada de credencial fica no repositório: o instalador perguntou na tela ou leu de uma variável de ambiente e gravou apenas no `.env`, que está no `.gitignore`.

---

2. Requisitos

Item	Mínimo	Como conferir
PHP	8.3	`php -v`
Extensões obrigatórias	pdo, mbstring, openssl, tokenizer, xml, fileinfo, curl, json	`php -m`
Extensões recomendadas	gd, intl, zip, exif, pdo_mysql	`php -m`
Composer	2.x	`composer --version`
MySQL	5.7 / MariaDB 10.3+	—
Apache com `mod_rewrite`	—	necessário quando o docroot é a raiz do projeto
Limite de envio	`upload_max_filesize` e `post_max_size` ≥ 40 MB	`php -i | grep upload_max`

O `gd` é o que permite ler a largura e a altura da imagem no campo de envio; sem ele o upload funciona, mas a dica de dimensão fica vazia. `intl` e `zip` são exigidos por pacotes do Composer; `exif` ajuda a validar foto de celular.

PHP 8.3 é inegociável

O `composer.json` pede `php: ^8.3` (Laravel 13). Se o servidor responder com

`Composer detected issues in your platform: Your Composer dependencies require a PHP version ">= 8.3.0". You are running 8.1.34.`

o site inteiro fica em erro 500, mesmo com banco e arquivos corretos. O ponto é que existem **dois PHPs no servidor**: o do comando `php` (linha de comando) e o que atende o navegador. Mudar um não muda o outro, e é o segundo que quebra o site.

- **cPanel:** *Software → MultiPHP Manager* → marque o domínio → *PHP Version*

→ escolha 8.3 (ou 8.4) → *Apply*. Depois confira em *MultiPHP INI Editor* os limites de `upload_max_filesize` e `post_max_size`.

- **cPanel com CloudLinux + LiteSpeed (o caso mais chato):** o *Select PHP

Version* da conta é um seletor próprio e pode parar no 8.2 mesmo com o 8.3 instalado no servidor. Nesse caso o `.htaccess` resolve, sim, desde que use o handler certo. Acrescente esta linha ao `.htaccess` da raiz do projeto (ou do `public/`, se o docroot for ele):

``` `apache AddType application/x-httpd-ea-php83 .php` ```

Foi exatamente assim que o 8.1 virou 8.3 num servidor cPanel + CloudLinux + LiteSpeed: o seletor da conta não oferecia 8.3 (`cloudlinux-selector get --interpreter php --get-supported-versions` parava em 8.2), mas o `ea-php83` estava instalado e o handler assumiu os arquivos `.php`. O instalador faz isso sozinho com `--php-web=ea-php83`.

Confira depois com um arquivo temporário dentro de `public/`:

``` `php <?php echo PHP_VERSION . ' | ' . PHP_SAPI; // salve como public/_chk.php` ```

Abra `https://seudominio/_chk.php`, confirme `8.3.x | litespeed` e **apague o arquivo** em seguida. Se aparecer `apachectl/lsapi` diferente ou a versão antiga, o handler escolhido não existe no servidor — liste os disponíveis com `ls -d /opt/cpanel/ea-php*`.

- **Extensões que costumam faltar no PHP da web:** `fileinfo` (upload de

imagem, vídeo e documento) e `intl`. No seletor do CloudLinux elas se acendem pela linha de comando, com o nome de estado por extensão — e a lista **substitui** a anterior, então informe tudo de uma vez:

``` `sh cloudlinux-selector set --json --interpreter php --version 8.3 --user SEU_USUARIO \ --extensions '{"fileinfo":"enabled","intl":"enabled","gd":"enabled","zip":"enabled","pdo_mysql":"enabled","mbstring":"enabled","bcmath":"enabled","dom":"enabled"}'` ```

- **Plesk:** *PHP Versions / Settings* → PHP 8.3.
- **VPS (Ubuntu):** `apt install php8.3-fpm php8.3-mysql php8.3-gd php8.3-intl php8.3-zip`

e aponte o `fastcgi_pass` do nginx para o 8.3.

O instalador detecta a versão do binário que você passou (`--php=`) e **para antes** de instalar qualquer coisa se ela for menor que 8.3. No final ele ainda abre o `--url` informado: se a home responder o erro de plataforma, ele avisa na tela que o PHP **da web** é outro e mostra a linha `--php-web=` que resolve.

---

3. Instalação com o instalador (recomendado)

Coloque o código no servidor (git ou upload do `.zip`), entre na pasta do projeto e rode:
```

```
GOLDEN_DB_PASSWORD='senha-do-banco' GOLDEN_ADMIN_PASSWORD='senha-do-painel' \
  bash deploy/instalador-golden.sh \
    --url=https://seudominio.com.br \
    --nome=nome_do_banco \
    --usuario=usuario_do_banco
```

Passar as senhas por variável de ambiente evita que elas fiquem no histórico do shell; sem as variáveis, o instalador pede na tela sem mostrar o que foi digitado.

Opções do instalador

Opção	Para que serve	
`--url=`	endereço público do site, com `https://` (obrigatório)	
`--banco=mysql|sqlite`	motor do banco; padrão `mysql`	
`--host=`, `--porta=`	servidor MySQL; padrão `127.0.0.1:3306`	
`--nome=`	nome do banco (no SQLite, o arquivo)	
`--usuario=`, `--senha=`	credenciais do banco	
`--criar-banco`	cria o banco se não existir (precisa de privilégio `CREATE`)	
`--admin-email=`, `--admin-nome=`, `--admin-senha=`	login do painel	
`--sem-demo`	não grava os dados de demonstração	
`--limpar-banco`	apaga todas as tabelas antes de recriar (sem a flag, ele pergunta)	
`--sem-backup`	não tira cópia do banco antes de limpar	
`--dev`	`APP_ENV=local` e `APP_DEBUG=true` (nunca em produção)	
`--sem-htaccess`	não escreve o `.htaccess` de proteção da raiz	
`--estatico=ARQUIVO`	mantém, na URL de origem, um arquivo que já está na raiz do domínio — repetível. É o que preserva a política de privacidade citada num app quando o Golden substitui um site antigo	
`--sem-cache`	não gera cache de configuração, rotas e views	
`--php=`, `--composer=`	força o binário quando há mais de uma versão no servidor	
`--php-web=ea-php83`	grava no `.htaccess` o handler do PHP que serve a web (CloudLinux/LiteSpeed)	
`--help`	tela de ajuda completa	

Exemplos prontos

cPanel, reaproveitando um banco que já existe no plano

```
GOLDEN_DB_PASSWORD='...' GOLDEN_ADMIN_PASSWORD='...' \
  bash deploy/instalador-golden.sh \
    --url=https://seudominio.com.br \
    --php=/usr/local/bin/ea-php83 --php-web=ea-php83 \
    --nome=usuario_do_banco --usuario=usuario_do_banco \
    --limpar-banco
```

VPS nova, criando o banco

```
bash deploy/instalador-golden.sh --url=https://painel.exemplo.com.br \
  --nome=golden --usuario=golden --criar-banco
```

Teste local, sem MySQL

```
bash deploy/instalador-golden.sh --url=http://localhost:8000 \
  --banco=sqlite --dev --sem-htaccess
```

O que aparece na tela

Cada etapa imprime uma linha com ✓. Se algo falhar, o instalador para, mostra o comando que falhou e as últimas 25 linhas do log (`storage/logs/instalador-*.log`), que guarda a saída completa. No fim ele imprime o resumo com endereço, painel, banco, ambiente e caminho do log.

Limpar um banco que já tem outro sistema

O instalador conta as tabelas antes de encostar. Havendo tabelas, ele:

1. faz `mysqldump` comprimido para a pasta **acima** do projeto — fora do que o

navegador serve — com permissão `600`;

1. pede confirmação explícita (digitar `SIM`), ou exige a flag `--limpar-banco`

quando roda sem terminal;

1. só então roda `migrate:fresh`.

Para restaurar a cópia depois:

```
gunzip < ../backup-banco-nome-20261023-1130.sql.gz | mysql -u USUARIO -p NOME_DO_BANCO
```

3.1 Trocar de domínio, ou substituir um site que já está no ar

Mudar de domínio não é configuração do sistema: o endereço público entra por `--url=` e nada mais. O que muda é a ordem dos passos, porque existe um site antigo morando naquela pasta.

1. **Cópia antes de encostar.** Compactar a pasta inteira do domínio para fora do

docroot (`~/backups-antes-do-golden/`, permissão `600`) e dumpar o banco que o site antigo usa — as credenciais estão no próprio arquivo de conexão dele. Nada é removido sem essa cópia conferida (contar arquivos e tabelas, não só rodar o comando).

1. **Retirar, não apagar.** Mover o conteúdo para uma pasta fora do docroot

preserva a volta atrás imediata. Ficam no lugar só o `.well-known` (renovação do certificado) e os arquivos marcados com `--estatico=`.

1. **Criar banco e usuário** pelo painel ou pela API da conta. Numa conta

cPanel sem privilégio de `CREATE USER`, o caminho é a API do próprio painel: criar o banco, criar o usuário e dar o privilégio com `Mysql::set_privileges_on_database` (argumentos `database`, `user`, `privileges=ALL PRIVILEGES`). Conferir com `show grants` antes de instalar.

1. **Instalar** com o endereço novo e a página antiga preservada:

```
bash deploy/instalador-golden.sh --url=https://novodominio.com.br \
  --php=/opt/cpanel/ea-php83/root/usr/bin/php --php-web=ea-php83 \
  --nome=usuario_banco --usuario=usuario_banco --estatico=privacidade.html
```

1. **Levar os dados** do outro domínio: dump do banco antigo (com

`--no-tablespaces` no cliente MySQL 8.4), importar sobre o banco novo e copiar `storage/app/public` — as imagens enviadas não viajam pelo dump. Um dump não atravessa de uma conta cPanel para outra (as pastas são `750`); ele precisa ser empurrado por quem tem as duas sessões.

1. **Refazer o resto do ambiente**: `storage:link`, caches, a linha de cron do

agendador apontando para a pasta nova e a caixa de e-mail de disparo, que é do domínio e precisa ser criada de novo.

---

4. Instalação manual, passo a passo

Serve para qualquer servidor e é o roteiro usado quando o instalador não pode rodar (por exemplo, hospedagem sem SSH).

4.1 Subir o código

- **Com git:** `git clone https://github.com/kdkhost/GOLDENRASTREAMENTO.git public_html`

- **Sem git:** baixe o `.zip` do repositório e extraia na pasta do domínio.

Confirme que `artisan`, `app/`, `public/` e `composer.json` estão na raiz da pasta extraída.

4.2 Criar o banco (cPanel)

1. *MySQL® Databases* → criar banco; a hospedagem costuma prefixar com o nome

da conta, ex.: `seuusuario_cms`. Anote o nome completo.

1. Criar usuário e **senha forte**; anote os dois.
2. Adicionar o usuário ao banco com **ALL PRIVILEGES**.
3. *phpMyAdmin* → conferir que o banco aparece e está vazio.

Se aparecer `Access denied for user`, faltou ligar o usuário ao banco.

4.3 Criar o `.env`

```
cp .env.example .env
php artisan key:generate
chmod 600 .env
```

Ajuste, no mínimo:

```
APP_ENV=production
APP_DEBUG=false
APP_URL=https://seudominio.com.br

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=seuusuario_cms
DB_USERNAME=seuusuario_cms
DB_PASSWORD=senha-do-banco
```

Veja a referência completa de todas as chaves na seção 7.

4.4 Dependências

```
composer install --no-dev --prefer-dist --optimize-autoloader
```

Sem Composer no PATH, baixe o arquivo único e use pelo PHP:

```
php -r "copy('https://getcomposer.org/installer', 'composer-setup.php');"
php composer-setup.php --install-dir=$HOME/bin --filename=composer.phar
php $HOME/bin/composer.phar install --no-dev --prefer-dist --optimize-autoloader
```

4.5 Banco de dados

```
php artisan migrate --force           # só as tabelas
php artisan migrate --force --seed    # com catálogo, páginas e demonstração
```

Para começar do zero em um banco que já tinha outra coisa:

```
php artisan migrate:fresh --seed --force
```

4.6 Imagens, vídeos e documentos

```
php artisan storage:link
```

Cria `public/storage` apontando para `storage/app/public`. Se o link não puder ser criado (algumas hospedagens bloqueiam o `symlink`), faça manualmente:

```
ln -s ../storage public/storage
```

Permissões: pastas `755`, arquivos `644`, e `storage/` + `bootstrap/cache/` precisam ser graváveis pelo usuário do webserver.

Sem esse link, **as fotos enviadas do painel não aparecem** — nem no site (`storage/site/…`, administrado em *Conteúdo → Imagens do site*), nem na loja e nos pedidos. O que não foi enviado pelo painel continua servindo de `public/img/`, então o site abre e só as fotos trocadas ficam quebradas: é o sintoma de `storage:link` faltando.

4.7 Entrega pelo docroot

Quando o domínio aponta para a **raiz do projeto** (caso típico do cPanel, `public_html`), use o `.htaccess` gerado pelo instalador — ele:

- desliga o índice de diretório (`Options -Indexes`), o que impede a listagem do

código-fonte;

- bloqueia `.env`, `.env.*`, `.git*`, `artisan`, `composer.*`, `package.json`,

`vite.config.js`, `phpunit.xml`, `error_log` e qualquer `.log`;

- bloqueia as pastas `app`, `bootstrap`, `config`, `database`, `resources`,

`routes` e `tests`;

- reescreve todo pedido para dentro de `public/`.

`storage` **não** entra na lista de bloqueio: o que o público alcança é o link `public/storage`, por onde saem as imagens, vídeos e documentos enviados pelo painel. Bloquear o caminho `/storage` na raiz devolve 403 em todo arquivo enviado, e o site fica com as imagens quebradas. A privateidade do `storage/` de verdade vem da reescrita (nada alcança `storage/logs`, `storage/framework` ou `storage/app/private`) e do bloqueio de `.log`.

O teste de barreira, já com o site no ar:

```
curl -I https://seudominio.com.br/.env          # 403
curl -I https://seudominio.com.br/app/          # 403
curl -I https://seudominio.com.br/storage/logs/laravel.log   # 403
curl -I https://seudominio.com.br/              # 200
curl -I https://seudominio.com.br/loja          # 200
```

E o teste do que o painel envia, que é o que costuma sobrar depois de mexer no `.htaccess`: envie uma imagem pelo painel e abra a URL devolvida no JSON de `/admin/uploads`. Tem de ser `200` com o `Content-Type` do arquivo — se der 403, a regra de bloqueio está alcançando `public/storage`.

Quando o docroot é a pasta `public/` (VPS com nginx, ou subdomínio apontando para `public`), o `.htaccess` da raiz nem é lido — e a proteção já existe por construção.

A entrega do que é estático mora em `public/.htaccess`, no repositório. O arquivo da raiz é gerado pelo instalador e cuida da *barreira*; o da pasta `public` cuida da *velocidade*, e por isso ele viaja com o código:

- CSS, JS, imagem e fonte saem com `Cache-Control: public, max-age=31536000` e

`Expires` de um ano; HTML sai com `Expires` zero, porque é página com sessão e formulário — quem cacheia a página perde o formulário.

- Cache de um ano é seguro porque cada arquivo é chamado com `?v=<filemtime>`:

mudou o arquivo, mudou o endereço na URL, e ninguém vê tela velha.

- `.woff2`, `.woff` e `.ttf` levam `Access-Control-Allow-Origin: *`. Fonte é

sempre pedida em modo CORS; no mesmo endereço isso não aparece, mas no dia em que o CSS sair por um subdomínio ou um CDN, sem a linha a tela inteira perde os ícones.

- `mod_deflate` comprime o que é texto. Onde o servidor já entrega brotli — é o

caso da hospedagem em OpenLiteSpeed — o brotli ganha e o bloco fica como par para o Apache puro de um servidor próprio.

O teste, depois de publicar:

```
curl -sI https://seudominio.com.br/css/golden-site.css | grep -i "cache-control\|expires"
curl -sI https://seudominio.com.br/webfonts/nao-existe   # 404, não 200 com HTML
curl -sI -H 'Accept-Encoding: br, gzip' https://seudominio.com.br/ | grep -i content-encoding
```

Em nginx a mesma regra vai para o bloco `location ~* \.(css|js|jpg|jpeg|png|webp|svg|ico|woff2)$ { expires 1y; add_header Cache-Control "public"; }` — o `.htaccess` é lido só por Apache e OpenLiteSpeed.

4.8 Caches de produção

```
php artisan config:cache
php artisan route:cache
php artisan view:cache
```

Depois de qualquer mudança em `.env`, `config/`, `routes/` ou `resources/views/` rode os três de novo (ou `php artisan optimize:clear` e recarregue).

4.9 Primeiro acesso

- Site: `https://seudominio.com.br`
- Painel: `https://seudominio.com.br/admin` — login definido na instalação.
- Portal do cliente: `https://seudominio.com.br/portal/entrar` — ninguém precisa

criar conta no painel; o auto-cadastro reaproveita a ficha do CRM (seção 10).

- Acessos de quem já é cliente: o ícone de login no topo do site abre o portal e o

financeiro antigo; o botão **Rastreamento** abre os dois servidores — na tela aparece só o nome de cada um, o endereço fica na ligação. Os três endereços externos se trocam em **Sistema → Configurações → Acessos externos do site** (o que a tela grava fica no banco e vence o `config`), junto do nome e do ícone de cada servidor. O cartão ao lado, **Textos dos acessos**, edita cada rótulo do caminho: o botão dourado, o ícone de conta, a área do cliente, as duas portas do submenu, o título e a descrição do modal e a frase de ajuda. Campo em branco devolve aquele rótulo ao padrão da casa.

- Troque a senha padrão assim que entrar, e remova o usuário de demonstração

(`vendedor@golden.com.br`) se não for usar.

---

5. Instalação no cPanel, em detalhe

1. *MultiPHP Manager* → PHP **8.3** para o domínio. Se o plano usar CloudLinux,

confira também o *Select PHP Version* da conta; quando ele não chega no 8.3, pule essa etapa e deixe o `.htaccess` escolher o handler (`--php-web=ea-php83`, seção 2).

1. *MySQL® Databases* → banco + usuário + privilégios (seção 4.2).
2. *Git™ Version Control* (se houver): *Clone Repository* com a URL pública do

repositório e o branch `main`, caminho `public_html`. Não use credencial na URL: o repositório é público. Sem o Git™, envie o `.zip` pelo *Gerenciador de Arquivos* e extraia.

1. *Terminal* (ou SSH com chave) → na pasta do projeto:

`git pull`, depois o instalador:

``bash GOLDEN_DB_PASSWORD='...' GOLDEN_ADMIN_PASSWORD='...' \ bash deploy/instalador-golden.sh \ --url=https://seudominio.com.br \ --php="$(command -v ea-php83)" --php-web=ea-php83 \ --nome=usuario_mysql --usuario=usuario_mysql --limpar-banco``

1. *Gerenciador de Arquivos* → confira as permissões: `.env` em `600`,

pastas `755`.

1. *MultiPHP INI Editor* → `upload_max_filesize=40M`, `post_max_size=45M`,

`max_execution_time=120` (o campo de vídeo aceita até 40 MB).

1. Abra o site e faça o teste de barreira da seção 4.7.
2. *SSL/TLS → Let's Encrypt* (ou o auto-SSL da hospedagem) ativo, e

Forçar HTTPS ligado.

Para outros agentes ou para usar de outra máquina, a mesma conta pode ser acessada por chave SSH: gere `ssh-keygen -t ed25519 -f ~/.ssh/golden_deploy -N ""`, cole a `.pub` em *Segurança → Chaves SSH / Access Hash*, e crie um apelido no `~/.ssh/config`. **Nunca** coloque senha, chave ou `.env` dentro do repositório.

---

6. Atualização depois da primeira instalação

```
bash deploy/atualiza-golden.sh                  # ramo main
bash deploy/atualiza-golden.sh --ramo=main --php=/usr/local/bin/ea-php83
```

Ele faz `git pull --ff-only`, `composer install --no-dev`, `migrate --force`, `storage:link` e reativa todos os caches. Se uma migration nova existir, ela é aplicada sem tocar nos dados.

Quando mexer em `.env` à mão:

```
php artisan config:clear && php artisan config:cache
```

7. Referência do `.env`

O `.env` é o plano de fundo, não o painel de controle. Todo valor configurável tem lugar nas telas — contato, loja, WhatsApp, medição, SMTP, chaves do Google Ads e do Business Profile se preenchem em **Sistema** → **Configurações** (as credenciais, na seção *Chaves de integração*, ao pé da página). O que o painel grava fica no banco de dados, cifrado quando é segredo, e passa na frente do ambiente no começo de cada requisição: salvou na tela, valeu na hora — sem `config:clear`, sem reiniciar nada. A tabela abaixo existe para o que ainda não foi preenchido em lugar nenhum, e para o que só faz sentido no servidor (banco, chave da aplicação, fila, fuso).

Chave	Produção	Observações
`APP_NAME`	`Golden Rastreamento`	nome em e-mails e no painel
`APP_ENV`	`production`	nunca `local` em servidor
`APP_DEBUG`	`false`	`true` vaza pilha de erro para o visitante
`APP_URL`	`https://seudominio.com.br`	usado em links de e-mail e assets
`APP_KEY`	gerado por `key:generate`	se trocar, sessões e dados criptografados somem
`APP_TIMEZONE`	`America/Sao_Paulo`	datas de pedidos e leads
`APP_LOCALE` / `APP_FALLBACK_LOCALE`	`pt_BR`	textos e moeda
`DB_CONNECTION`	`mysql`	`sqlite` apenas em teste
`DB_HOST`, `DB_PORT`	`127.0.0.1`, `3306`	no cPanel costuma ser `localhost`
`DB_DATABASE`, `DB_USERNAME`, `DB_PASSWORD`	do painel	nunca no git
`SESSION_DRIVER`	`file`	sessões não geram consulta ao banco
`CACHE_STORE`	`file` (ou `redis`)	
`QUEUE_CONNECTION`	`database`	exige `php artisan queue:work`
`FILESYSTEM_DISK`	`public`	onde os envios vão parar
`LOG_CHANNEL` / `LOG_LEVEL`	`stack` / `warning`	log em `storage/logs`
`MAIL_MAILER`	`smtp`	use `log` enquanto não tiver credencial
`MAIL_LOG_CHANNEL`	`email`	para onde o `log` escreve o e-mail (`storage/logs/email.log`)
`MAIL_HOST`, `MAIL_PORT`, `MAIL_USERNAME`, `MAIL_PASSWORD`, `MAIL_ENCRYPTION`	do seu servidor de correio	o **código de e-mail está pronto no projeto**; a credencial se preenche na tela (seção 7.1), e estas linhas ficam como plano de fundo

Chave	Produção	Observações
`MAIL_FROM_ADDRESS`, `MAIL_FROM_NAME`	`disparos@foco.server-golden.com.br` / `Golden Rastreamento`	remetente dos orçamentos e avisos: caixa de máquina, não o e-mail de uma pessoa
`GOLDEN_WHATSAPP`	`5519997101483`	botão de WhatsApp do site
`GOLDEN_EMAIL`	endereço comercial	aparece no site e nos avisos
`GOLDEN_TRACKING_ENABLED`	`0` ou `1`	liga a consulta de rastreio
`GOLDEN_FINANCEIRO_URL`, `GOLDEN_SERVIDOR_1_URL`, `GOLDEN_SERVIDOR_2_URL`	do seu ambiente	portas de entrada do cabeçalho do site (financeiro antigo e os dois servidores); **preencha em Sistema → Configurações → Acessos externos do site**, o `.env` é só plano de fundo
`GOOGLE_BUSINESS_ACCOUNT`, `GOOGLE_BUSINESS_LOCATION`, `GOOGLE_BUSINESS_TOKEN`	do seu perfil	depoimentos do Google com avatar; **preencha na tela** (seção 11), o `.env` é só plano de fundo
`GOOGLE_BUSINESS_LIMITE`, `GOOGLE_BUSINESS_NOTA_MINIMA`	`50`, `4`	quantas avaliações ler por rodada e a nota mínima aceita; também ajustáveis na tela
`GOOGLE_ADS_DEVELOPER_TOKEN`	da sua MCC	token de desenvolvedor da API; **segredo da aplicação**, nunca sai para a tela (seção 9.1)
`GOOGLE_CLIENT_ID`, `GOOGLE_CLIENT_SECRET`	do Google Cloud	app OAuth da tela de consentimento; o painel conecta conta por conta com eles
`GOOGLE_REDIRECT_URI`	`${APP_URL}/auth/google-ads/callback`	tem de ser o URI cadastrado no Google, endereço exato
`GOOGLE_ADS_LOGIN_CUSTOMER_ID`	MCC sem hífen	vazio é legítimo: cada conta conectada é consultada por si só
`GOOGLE_ADS_TRANSPORT`	`rest`	`grpc` só onde a extensão do PHP existe (servidor próprio)
`GOOGLE_ADS_CACHE_STORE`	vazio (= `CACHE_STORE`)	escreva `redis` só se houver **servidor** Redis, não apenas a extensão

Chave	Produção	Observações
`GOOGLE_ADS_CACHE_MINUTOS`, `GOOGLE_ADS_DIAS`	`30`, `30`	vida da janela sincronizada e período padrão da tela

Sobre os e-mails: os modelos, o envio de orçamento e as notificações de pedido já existem. Trocar `MAIL_MAILER=log` para `smtp` e preencher as cinco chaves é tudo que falta para eles saírem de verdade. Em `log`, a mensagem inteira é escrita em `storage/logs/email.log` — útil para conferir o conteúdo antes de liberar o SMTP. Esse canal é separado do log da aplicação de propósito: o driver `log` grava o aviso em nível `debug`, e uma produção com `LOG_LEVEL=warning` filtrava essas linhas — o `queue:work` dizia `DONE` e não sobrava rastro nenhum do envio.

A caixa de disparo se cria pela SSH, sem abrir o painel

Não use o e-mail de ninguém do dia a dia como remetente do sistema: crie uma caixa de máquina, só para conexão e disparo. Com SSH na hospedagem cPanel:

```
uapi Email add_pop email=disparos@seudominio.com.br password='SENHA_FORTE' quota=0
uapi Email list_pops          # confere que a caixa existe
```

Três coisas que só aparecem na hora:

- **o domínio da caixa tem de ser um que a conta hospeda.** A API responde

`you do not have access to the domain 'outrodominio.com.br'` para domínio que está em outra conta do mesmo servidor. `uapi Email list_mail_domains` lista os liberados.

- **SPF e DKIM precisam cobrir esse domínio.** Confira `TXT seudominio.com.br` e

`TXT default._domainkey.seudominio.com.br`; sem isso a mensagem sai e cai em spam.

- **senha com `#` no `.env` só funciona entre aspas.** Sem as aspas, o `#` começa um

comentário e o `MAIL_PASSWORD` chega vazio ao Laravel — o erro aparece como falha de login no SMTP, não como erro de arquivo.

Depois, preencha a caixa em **Sistema → Configurações → Chaves de integração → E-mail de disparo** (host, porta, criptografia, usuário, senha e remetente) e escolha "SMTP (servidor de correio)" em "Como o sistema envia". A senha cifra no banco e não volta a aparecer na tela. Gravou, vale — e uma prova curta antes de mexer no site:

```
Mail::raw('prova', fn ($m) => $m->to('voce@seudominio.com.br')->subject('prova'));
```

Se o envio for aceito, a mensagem aparece na caixa de destino; o `queue:work` do cron (seção 6) cuida do resto sem intervenção. Escrever as mesmas chaves no `.env` continua funcionando, como plano de fundo — é o caminho de quem ainda não abriu o painel.

---

8. Loja virtual

A loja (catálogo, carrinho, checkout e pedidos no painel) vem ligada. Para desligar a vitrine sem desligar o resto, use o painel em *ERP → Configurações da loja* (chaves `loja_ativa` e `loja_livre`); o site some do menu e o `/loja` responde 404, mas os pedidos já registrados continuam no painel.

Os campos de envio do painel (imagem do produto, vídeo do produto, documentos do pedido e o PDF do contrato) aceitam arrastar-e-soltar, mostram preview, progresso real, tempo decorrido e a regra do campo (formatos, tamanho máximo e dimensão ideal). As regras moram em `config/envios.php`:

Campo	Formatos	Limite	Pasta no disco
`produto_imagem`	JPG, JPEG, PNG, WEBP	4 MB	`produtos/`
`produto_video`	MP4, WEBM	40 MB	`produtos/video/`
`pedido_documento`	PDF, JPG, JPEG, PNG	10 MB cada, até 12 por pedido	`pedidos/`
`contrato_documento`	PDF	8 MB	`contratos/`

O servidor confere a extensão **do conteúdo** do arquivo, não o nome que o navegador mandou, e recusa o envio se o tipo não estiver entre os formatos do campo. Por isso os limites de `upload_max_filesize` / `post_max_size` da seção 2 importam: arquivo maior que isso chega cortado e o campo avisa o limite do servidor.

---

9. Tráfego e atribuição

O sistema identifica a origem de **cada visita, clique e cadastro** sem depender de Google Analytics: as tabelas `visits` e `visit_events` gravam a jornada, e o painel abre em *Comercial → Atribuição de origens* (`/admin/atribuicao`) com as tabelas por canal, rede, campanha, página e dispositivo; a ficha do lead (`/admin/leads/12`) mostra primeiro toque, último toque e a linha do tempo da visita.

Nada precisa ser configurado para funcionar: os sinais vêm da URL (`utm_*`, `gclid`, `fbclid`, `ttclid`, `msclkid`, `li_fat_id`, `yclid`), do referer e do user-agent. As listas que mandam na classificação estão em `config/golden.php` (`canais`, `meios_pagos`, `meios_sociais`, `meios_ia`, `buscadores`, `ia_usuarios`, `indexadores`) — acrescentar uma rede nova é editar o config, não o código.

O cookie de visitante (`golden_visitor`, 90 dias) é de primeira parte e não guarda dado pessoal; o IP entra como hash. Robôs de índice (`GPTBot`, `ClaudeBot`, `Meta-ExternalAgent`, o crawler do Google…) entram no registro mas ficam **fora de todo número** — já o navegador embutido do WhatsApp/Instagram, onde o clique humano aconteceu, conta como gente.

Retenção e agendamento

O registro cresce com o tráfego, por isso há poda:

```
php artisan rastreio:podar            # usa golden.rastreio.dias_retencao (365 dias)
php artisan rastreio:podar --dias=90  # período menor, quando preciso
```

A poda é uma tarefa da agenda do painel (`Sistema → Crons`), semeada na instalação para toda terça às 03:30. O agendador do Laravel lê a tabela `cron_tarefas` e precisa de **uma** linha no cron da hospedagem (cPanel → *Tarefas de Cron*):

```
* * * * * cd ~/public_html && /opt/cpanel/ea-php83/root/usr/bin/php artisan schedule:run
>> /dev/null 2>&1
```

O caminho do PHP é o **8.3 do site**, não o `php` do PATH da conta (que costuma ser mais velho). Para descobrir o certo no seu servidor: `ea-php83 -r 'echo PHP_VERSION;'` ou `ls /opt/cpanel/ea-php8*/root/usr/bin/php`. Sem o cron, o comando continua podendo ser rodado à mão — só não se auto-agenda.

A mesma linha de cron também drena a fila de e-mails. Todo aviso do portal (chamado aberto, resposta da equipe, recibo, link de recuperação de senha) entra em fila, e a tarefa **Drenar a fila de envios** — uma das quatro que a instalação semeia — dispara `queue:work --stop-when-empty --max-time=50` a cada minuto. Numa hospedagem sem esse cron as mensagens ficam paradas na tabela `jobs` e o cliente espera um e-mail que nunca sai. Para conferir à mão:

```
php artisan queue:work --stop-when-empty   # consome o que está preso e sai
```

Não use `queue:listen` nem deixe um worker residente: a hospedagem compartilhada não tem supervisor, e o `--stop-when-empty` existe justamente para o trabalho caber no disparo de um minuto do `schedule:run`.

Se um aviso estourar ao ser montado (endereço inválido, dado faltando), ele não se perde: sai de `jobs` e entra em `failed_jobs`, com o erro por extenso. Depois de corrigir a causa, o retry recoloca a mensagem na fila sem precisar abrir o chamado de novo:

```
php artisan queue:retry all    # devolve tudo que está em failed_jobs para a fila
```

9.1 Google Ads no painel

Comercial → Google Ads (`/admin/google-ads`) mostra a métrica real das campanhas do anunciante dentro do painel: impressões, cliques, custo, conversões, CTR médio e ROAS, com a tabela de campanhas filtrável por período (7/30/90 dias), situação e busca pelo nome. *Comercial → Criativo do anúncio* (`/admin/google-ads/criativo`) é o editor do anúncio responsivo com a prévia do resultado da busca.

Cada pessoa que entra no painel conecta **a própria conta** pelo OAuth do Google. A conexão fica em `google_ads_accounts` com o token de renovação cifrado no banco, e cada chamada devolvida pela API fica em `google_ads_api_logs` (endpoint, requisição, resposta, erro) para auditoria. O administrador vê as conexões da equipe inteira — a agência opera as contas dos clientes —; qualquer outro papel vê só as que ele conectou.

O que a tela nunca faz

Nenhuma requisição do painel chama a API do Google. A tela lê a janela de cache `google_ads:metrics:{usuário}:{conta}:{dias}` (30 minutos de vida) e, se ela está vazia, enfileira `SyncGoogleAdsMetrics` e mostra "Carregando..." com verificação automática a cada 3 segundos. Por isso um F5 rápido não custa chamada de rede, não estoura a cota da API e não deixa ninguém esperando pelo tempo de resposta do Google.

Quem preenche o cache é a fila: o job tem três tentativas com espera de 30 s, 2 min e 5 min, e o agendamento `google-ads:sync` roda a cada 15 minutos para todas as contas ativas. Quando o Google **recusa a credencial** (token revogado, senha da conta alterada), o job não insiste: marca a conta como desconectada, escreve o motivo na tela e avisa o dono por e-mail.

Passos que só existem no Google (uma vez por conta gerente)

Nenhum deles é trabalho de código, e sem eles a tela mostra exatamente o que falta:

1. **Conta MCC (gerenciada).** Em *Google Ads → Ferramentas e configurações → Estrutura de

contas*, crie a conta gerente e anexe as contas de anunciante a ela. O número da MCC, **sem hífens**, é o `GOOGLE_ADS_LOGIN_CUSTOMER_ID`. Sem MCC o campo fica vazio e cada conta conectada é consultada por si só.

1. **Token de desenvolvedor.** Na MCC: *Ferramentas e configurações → API do Google Ads →

Aplicação de teste*. Preencha o nome da aplicação, aceite os termos e peça o acesso (padrão **Basic Access**; para escrever, o Google pode pedir revisão). O token gerado é o `GOOGLE_ADS_DEVELOPER_TOKEN`. **É segredo da aplicação, não do anunciante**: nunca aparece no navegador, em view, JSON ou log.

1. **Tela de consentimento OAuth + credenciais.** No Google Cloud console,

crie um projeto, configure a *tela de consentimento OAuth* (tipo **Externo**, com o domínio nos *domínios autorizados* e `contato@kdkhost.com.br` como contato de desenvolvimento) e crie uma credencial do tipo **Aplicação da Web**: - *URI de redirecionamento autorizado*: `https://SEU_DOMINIO/auth/google-ads/callback` — o endereço exato, sem barra no fim; mudar o endereço é refazer o cadastro. - O `GOOGLE_CLIENT_ID` e o `GOOGLE_CLIENT_SECRET` que a credencial devolve. - O escopo pedido pelo painel é `https://www.googleapis.com/auth/adwords`, o único que a API aceita; nada de escopos genéricos do Google.

1. **Ativar a API do Google Ads** no mesmo projeto (*Biblioteca* → "Google Ads API" → Ativar).

As cinco credenciais: a tela, não o arquivo

Token de desenvolvedor, ID e segredo do cliente OAuth, endereço de volta e MCC se preenchem em **Sistema → Configurações → Chaves de integração → Google Ads** (`/admin/configuracoes#chaves`). O que for gravado ali entra no banco — os três segredos cifrados, e nenhum deles volta a aparecer na tela depois de salvo — e vale na hora: não há `config:clear`, nem `artisan` nem reinício de servidor no meio. Os nomes de `.env` continuam existindo como **plano de fundo**, para o campo que ninguém preencheu; deixá-los todos em branco no arquivo é o estado normal de uma instalação que configura pela tela.

Na tela	A variável de plano de fundo
Token de desenvolvedor	`GOOGLE_ADS_DEVELOPER_TOKEN`
ID do cliente OAuth	`GOOGLE_CLIENT_ID`
Segredo do cliente OAuth	`GOOGLE_CLIENT_SECRET`
Endereço de volta do OAuth	`GOOGLE_REDIRECT_URI` (`${APP_URL}/auth/google-ads/callback`)
Conta gerente (MCC)	`GOOGLE_ADS_LOGIN_CUSTOMER_ID` — vazio é legítimo

Restam no arquivo duas linhas de hospedagem, e deixar os padrões é o certo nela:

- `GOOGLE_ADS_TRANSPORT=rest` — a biblioteca oficial fala com o Google por gRPC **ou** REST.

A hospedagem compartilhada não tem root nem `pecl`, então a extensão `grpc` do PHP não entra. Numa máquina sua com a extensão, troque para `grpc`: nenhuma linha de código muda.

- `GOOGLE_ADS_CACHE_STORE=` vazio usa o store do próprio app. A hospedagem tem a extensão

Redis do PHP mas **não tem o servidor Redis**, então a métrica fica no `file`. Com um servidor Redis no ar, escreva `redis` e a fila inteira passa a usar o mesmo ponto.

Rodar tudo

```
php artisan google-ads:sync                # enfileira todas as contas ativas
php artisan google-ads:sync --conta=3      # uma conexão específica
php artisan google-ads:sync --dias=7       # janela diferente do padrão
```

O comando **enfileira**; quem chama a API é o worker. Numa hospedagem sem supervisor, o cron do `schedule:run` (seção 9) já drena a fila a cada minuto — é a mesma linha que serve aos e-mails.

Escrever na conta (pausar e ativar)

Os botões da tabela pausam e reativam campanha, e cada escrita passa por **duas** chamadas: primeiro `validarOperacao`, que envia a mesma operação com `validate_only => true` e percorre permissão, status e FieldMask sem tocar em nada; só depois, se o Google aprovou, `mutateCampaigns` escreve de verdade. Uma conta real de anunciante não é lugar de experimentar.

A escrita nunca é chamada pelo Livewire: é formulário POST com confirmação em SweetAlert2 e retorno em toastr, como todo ato destrutivo do painel. O `campaign_id` não vem do formulário como verdade — é conferido contra a lista sincronizada **daquela conta**, para o painel não escrever num número que alguém digitou ou que pertence a outra conta conectada.

Prévia do anúncio

Não existe API oficial de prévia de anúncio responsivo; o mock em HTML/CSS é o padrão do mercado. O editor aplica as travas do Google — até 15 títulos de 30 caracteres, até 4 descrições de 90, duas rotas de display com trava **por segmento** — com contador por campo (verde, âmbar, vermelho) e três combinações de título/descrição no formato da busca, em desktop e celular. As combinações são as que o Google pode montar; ele escolhe, e a prévia mostra o corte real de cada uma.

Se a tela disser que falta alguma coisa

O que aparece	O que é
"A integração ainda não tem credenciais: faltam …"	Campo vazio na tela **e** no arquivo; a lista diz quais. Preencha em Sistema → Configurações → Chaves de integração — vale na hora.
"Conectar conta Google Ads" sumiu / erro do OAuth	`GOOGLE_REDIRECT_URI` diferente do URI cadastrado no Google Cloud.
"Este token não alcança nenhuma conta de anunciante"	Consentimento dado sem conta ativa na MCC, ou conta nova com menos de 24 h. Reconecte.
"Carregando…" por mais de alguns minutos	O worker não está rodando: `php artisan queue:work --stop-when-empty` à mão e confira o cron do `schedule:run`.
Selo **Desconectada** com motivo	O Google recusou o token de atualização. Reconecte; o histórico sincronizado continua na tela.

Nada disso precisa ser procurado neste documento enquanto a tela está aberta: os quatro passos, as cinco chaves, os comandos e esta mesma tabela vivem no cartão dobrável **"Como configurar"** da própria tela de Google Ads, que **nasce aberto enquanto faltar credencial** e fica fechado quando não há mais o que explicar. O guia inteiro está na barra lateral do painel, em **Guia de uso** (seção 14).

---

10. Portal do cliente

O portal (`/portal`) é a casa do cliente no mesmo domínio do site: ele vê os próprios pedidos, abre e acompanha chamados, lê os contratos publicados e corrige o cadastro — sem depender da equipe e sem nunca receber acesso ao painel.

Tela	Endereço
Entrar	`/portal/entrar`
Criar conta (auto-cadastro)	`/portal/cadastro`
Esqueci a senha / redefinir	`/portal/esqueci-senha`, `/portal/redefinir-senha`
Início (resumo)	`/portal`
Pedidos	`/portal/pedidos`, `/portal/pedidos/{id}`
Chamados	`/portal/chamados`, `/portal/chamados/abrir`
Contratos	`/portal/contratos`
Meus dados	`/portal/dados`

A entrada aparece no menu do site, na gaveta do celular e na coluna *Suporte* do rodapé, sempre com o mesmo link: `/portal/entrar` para quem ainda não entrou e `/portal` para quem já tem sessão.

A conta nasce sozinha

Ninguém precisa cadastrar cliente no painel para ele entrar. O auto-cadastro procura a ficha do CRM primeiro pelo **documento** (com ou sem máscara) e depois pelo **e-mail**; achou, entrega a senha a ela, e a ficha ganha as quatro colunas de acesso (`email_portal`, `password`, `remember_token`, `ultimo_acesso_em`). Entre duas candidatas, a que tem mais pedidos ganha. Não achou nada, cria a ficha nova, já ativa — e os pedidos feitos como convidado na loja são puxados pelo e-mail, então a primeira tela já vem cheia.

Isso é o que mais gera chamado de suporte: o login do portal é o `email_portal`, e não o e-mail da ficha do CRM. São colunas diferentes de propósito, porque o CRM aceita e-mail repetido entre fichas e a conta do portal não. A ficha do painel (*Clientes → qualquer ficha*) mostra o selo *conta ativa* com o e-mail de acesso e o último acesso, ou *sem conta* — é esse e-mail que funciona no `/portal/entrar`.

Sessão separada, por construção

O portal autentica na guarda `cliente` sobre a tabela `customers`; o painel, na guarda padrão sobre `users`. As duas sessões têm chaves próprias (`login_cliente` / `login`), então abrir o portal não abre rota do painel nem o contrário — dá para ter as duas no mesmo navegador, logadas em pessoas diferentes. Recurso de outro cliente (pedido, chamado ou contrato por id) devolve **404, não 403**: a barra de endereços não descobre o que existe.

O que a equipe faz com isso

- **Admin → Atendimento → Chamados**: KPIs de abertos, sem resposta, aguardando

o cliente e resolvidos; filtro por status e prioridade; responsável; resposta e mudança de status na tela do chamado. A conversa é uma fila só (`tickets` + `ticket_messages`) — a primeira mensagem é o relato que abriu o chamado —, então não há "última resposta" para sincronizar entre as duas telas.

- **Admin → Atendimento → Contratos**: cadastra o documento, anexa o PDF assinado

no dropzone (campo `contrato_documento`: PDF, 8 MB, pasta `contratos/`) e controla a visibilidade pelo status. **Rascunho não aparece para o cliente** — nem na lista, nem no resumo, nem no download. Número comercial `CT26-0001`, vigência com aviso de vencimento em 30 dias.

O PDF do contrato **não** sai por URL pública de `/storage`: o download passa por `/portal/contratos/{id}/arquivo`, que confere se o contrato é da ficha logada e se não é rascunho antes de entregar. Um caminho de arquivo escrito à mão no formulário é recusado — a gravação revalida contra a pasta do campo de envio.

E-mails e recuperação de acesso

Quatro avisos saem do portal: `ChamadoRecebido` (para o cliente), `ChamadoEquipe` (para o painel), `ChamadoResposta` (bidirecional) e `RedefinirSenhaPortal`. Como no resto do sistema, **sem SMTP configurado o aviso se perde e a ação do cliente nunca é barrada** — o envio é tentado em `try/catch`; com `MAIL_MAILER=log` a mensagem fica em `storage/logs/email.log`.

A recuperação de senha não usa tabela de tokens nem broker: o link é assinado (`URL::temporarySignedRoute`) e expira em 60 minutos. Consequências práticas:

- a assinatura depende da `APP_KEY` — trocar a chave invalida links já enviados,

o que é o comportamento desejado, não um bug;

- o e-mail com ou sem conta recebe **a mesma frase**, para a tela não revelar quem

é cliente da Golden;

- o throttle é próprio por rota (cadastro `8,1`, login `10,1`, recuperação `6,1`,

abrir chamado `12,1`) e o cadastro tem honeypot (`website`) mais consentimento LGPD obrigatório.

Migrações novas

Duas migrations entram na atualização: `2026_10_24_000001_cria_conta_do_portal` (as quatro colunas em `customers`) e `2026_10_24_000002_create_tickets_and_contracts_tables` (`tickets`, `ticket_messages`, `contracts`). São aditivas e cada uma tem `down()`; `php artisan migrate --force` as aplica sem tocar nos dados existentes. Toda coluna de texto tem comprimento explícito e `email_portal` fica em 190 — o MySQL da hospedagem não aceita índice único maior, e o SQLite dos testes não avisa.

---

11. Depoimentos do site: painel, Google e portal

O carrossel da home é uma lista só — a tabela `testimonials` — e cada linha diz de onde veio no campo `fonte`:

`fonte`	Quem escreve	Vai ao ar?	Foto do autor
`site`	a equipe, em *Conteúdo → Depoimentos*	quem digita decide	upload opcional no formulário
`google`	`php artisan depoimentos:google` ou o botão **Atualizar do Google**	entra aprovado, com a nota da avaliação	baixada do perfil para `storage/app/public/site/depoimentos`
`cliente`	o próprio cliente, no portal (*Deixar um depoimento*)	**sempre nasce oculto**, aguardando a equipe	a inicial do nome

No painel as três aparecem misturadas na mesma tela, com o selo da origem, o filtro **Origem** e um aviso no topo contando quantos ainda aguardam aprovação. No site, `Testimonial::aprovadosParaSite()` só entrega quem tem `aprovado = true`, ordenado por `ordem` e depois pela data — é por isso que a mesclagem não tem chave nem configuração: aprovar é o único interruptor.

Ler as avaliações do Google

A fonte é a *Business Profile API v4* (`accounts/<ID>/locations/<ID>/reviews`) — a única que devolve a foto de quem avaliou; o Places API comum só entrega avaliações para conta Enterprise. Antes de funcionar, três coisas precisam existir:

1. o perfil verificado no Google Business Profile há mais de 60 dias;
2. a *Business Profile API* liberada para o seu projeto no Google Cloud — a

aprovação do Google leva dias úteis;

1. um Bearer token de OAuth de uma conta que administra esse perfil, com o escopo

`https://www.googleapis.com/auth/business.manage`. Com o `gcloud` instalado:

```
gcloud auth application-default login --
scopes=https://www.googleapis.com/auth/business.manage
gcloud auth application-default print-access-token
```

As três peças se preenchem em **Sistema → Configurações → Chaves de integração → Avaliações do Google**, e vão cifradas ao banco — o token nunca aparece em tela, log nem repositório. Para quem prefere o servidor, as mesmas chaves existem no `.env` como plano de fundo:

Nome na tela	Chave de plano de fundo	Valor
Conta do Business Profile	`GOOGLE_BUSINESS_ACCOUNT`	o ID da conta, `accounts/123456789` (ou só `123456789`)
Local (ficha)	`GOOGLE_BUSINESS_LOCATION`	o ID da unidade, `locations/987654321` (ou só `987654321`)
Token de acesso	`GOOGLE_BUSINESS_TOKEN`	o Bearer de OAuth — expira e precisa ser renovado
Avaliações por rodada	`GOOGLE_BUSINESS_LIMITE`	avaliações lidas por rodada (padrão `50`)
Nota mínima para virar depoimento	`GOOGLE_BUSINESS_NOTA_MINIMA`	nota mínima que vira depoimento (padrão `4`)

Com elas preenchidas:

```
php artisan depoimentos:google --apenas-leia   # mostra o que a API devolve, sem gravar
php artisan depoimentos:google                 # importa e atualiza
php artisan depoimentos:google --limite=25
```

A importação é **idempotente**: cada avaliação fica guardada pelo `reviewId` na coluna `externo_id`, então rodar de novo atualiza texto, nota e foto em vez de criar um segundo depoimento da mesma avaliação. Quem não tem texto não entra (sai na contagem `sem_texto`), nem quem está abaixo de `GOOGLE_BUSINESS_NOTA_MINIMA` — nota baixa é chamado, não vitrine. E o importado que a equipe desligou no painel **não reacende** na rodada seguinte.

Sem as chaves nada quebra: o comando e o botão dizem **qual campo falta** e apontam a tela de Chaves de integração, em vez de chamar a API no escuro. Quando o token expira, o painel devolve o aviso vermelho com a resposta do Google e a linha `depoimentos_google_recusados` fica em *Auditoria*. Para manter o carrossel fresco sem ninguém precisando lembrar, o comando vira uma tarefa da agenda do painel — **Sistema** → **Crons**, botão *Nova tarefa*, comando `depoimentos:google --limite=50` e frequência semanal. Nada é escrito na crontab da hospedagem: a única linha dela é a do `schedule:run` (seção 9), e a agenda lê a tabela `cron_tarefas` (seção 15).

Depoimento escrito pelo cliente

No formulário do painel, o campo **Ficha do cliente** lista apenas fichas com conta no portal (`email_portal` preenchido): é ele que faz a linha dizer quem escreveu, do mesmo jeito que o portal faz quando o cliente envia de dentro da própria conta. O portal abre a tela *Deixar um depoimento* para todo cliente autenticado, mas só grava quando a ficha está `status = ativo`; enquanto houver um texto do mesmo cliente aguardando aprovação, a tela avisa em vez de acumular duplicados. O envio nasce `fonte = cliente`, `aprovado = false` e com o `customer_id` assinando — e a equipe recebe o aviso no `contato_email` já apontando para a fila de pendentes.

Migração nova

`2026_10_24_000003_da_origem_ao_depoimento` acrescenta `fonte` (16, default `site`), `externo_id` (191, único), `customer_id` (FK para `customers`, zera se a ficha for removida) e `avatar` (500). É aditiva, tem `down()`, e os depoimentos que já existiam recebem `fonte = 'site'` pelo próprio default da coluna.

---

12. Vídeos tutoriais na galeria do site

A seção de vídeos do site antigo volta como página própria: `/videos`, com o cartaz editável do CMS, o filtro por categoria e o player abrindo na mesma tela. O conteúdo é a tabela `videos`; cada linha é um tutorial do YouTube.

Coluna	O que guarda
`titulo`	o nome do tutorial, até 160 caracteres
`codigo`	os **11 caracteres** do YouTube — únicos, e é deles que saem miniatura, link e player
`categoria`	uma das chaves de `config('golden.video_categorias')`
`descricao`	opcional, até 500 caracteres
`ordem`	define a sequência na galeria e na listagem do painel
`ativa`	desligada esconde o vídeo do site sem apagar o registro

Cadastrar pelo painel

Conteúdo → Vídeos tutoriais → Novo vídeo. No campo **Link ou código do vídeo** cola-se exatamente o que o navegador copia — `watch?v=`, `youtu.be`, `shorts/`, `embed/`, `live/` ou o código sozinho. O sistema extrai o código de 11 caracteres antes de gravar; se não houver código ali, a mensagem diz isso em português e o texto colado continua no campo para corrigir. O mesmo código não entra duas vezes. A miniatura que o site vai mostrar aparece no card da direita, antes de salvar.

Categorias, rótulos, ícones e cores ficam em dois lugares do mesmo arquivo (`config/golden.php`): `video_categorias` e `tons_status.video`. Criar uma categoria nova é acrescentar uma linha em cada bloco — não há tabela de categorias nem second CRUD para manter.

Como a página serve

`Video::paraSite()` monta a galeria inteira no cache `golden.videos_site`, então abrir `/videos` não faz SELECT; gravar, editar ou excluir no painel derruba o cache na hora (`CachesContent`). O filtro por categoria é link verdadeiro (`?categoria=frotas`): funciona sem JavaScript, é indexável e não precisa de biblioteca de filtragem no cliente.

A galeria é paginada em **12 por página** (`config('golden.videos_por_pagina')`, régua única — a tela não escolhe número próprio), com resumo "1–12 de 24 tutoriais" e pager logo abaixo da grade. A fatia é feita **em**

memória, sobre o cache do modelo: paginar consultando o banco a cada página jogaria fora o que faz a tela abrir sem nenhum SELECT. Pedir uma página além da última (`?page=99`) volta para a última real em vez de erro ou tela em branco, e o pager carrega a categoria escolhida em cada link.

O pager do site é um view próprio (`resources/views/pagination/site.blade.php` , seção 18) porque nem o do Tailwind nem o do painel vestem o CSS do site. As miniaturas vêm de `i.ytimg.com` , e a página adianta a conexão com `preconnect` — só ela, já que é a única que usa.

O `<iframe>` só existe depois que o visitante abre um cartão, e usa `youtube-nocookie.com` (nada é gravado em quem só passou os olhos pela galeria). Abrir **não** põe o vídeo para tocar: o quadro nasce coberto pelo vidro da casa e o play é sempre um toque seguinte em controle nosso. Fechar o modal pausa e volta o quadro ao zero — é isso que corta o áudio e é isso que não deixa a tela final do YouTube na frente de ninguém. Cada clique conta "Abriu um tutorial em vídeo" na jornada do visitante (evento `video` de `config('golden.rastreio.eventos')`).

A galeria tem cartaz próprio (`hero_videos`), entrada no menu institucional, no rodapé e um link na seção de dúvidas da home — a home não repete os cartões.

O player da casa é uma peça só

`resources/views/site/partials/video-player.blade.php` é o único lugar do projeto que descreve um quadro de vídeo. A faixa da campanha e o modal da galeria incluem o mesmo partial, então o comportamento é idêntico nas duas telas e a regra mora em um lugar só: o `montarVideo()` de `public/js/golden-site.js` . O `public/js/golden-funil.js` que a campanha tinha foi excluído, e nenhum `.blade.php` escreve `<iframe>` — quem monta o player é a API, sobre o nó `.video-quadro` .

As quatro invariantes, cada uma com teste travando:

- **nada toca sozinho.** `autoplay: 0` , `cueVideoById` ao trocar de vídeo (nunca

`loadVideoById` , que já sai tocando) e abrir quadro é só abrir. O toque dado antes de a API chegar é guardado e aplicado no `onReady` , porque ali quem pediu foi o visitante;

- **nada diz YouTube na tela.** Nenhum botão, marca ou texto do YouTube é escrito pela

casa, e o `title` do iframe é o rótulo da faixa ou o título do tutorial — costurado de volta por um `MutationObserver` , porque a API reescreve o atributo quando recebe os metadados do vídeo. A única exceção é invisível e é o plano B: o `href` do card da galeria aponta para `youtube.com/watch?v=…` para que, sem JavaScript, o link continue funcionando — com JavaScript ele nunca chega a ser seguido;

- **o vidro cobre enquanto não toca.** `.video-vidro.cobre` é opaco (`#05070a`) e

traz o selo dourado de play; ele levanta no primeiro frame tocando e desce de novo ao pausar, ao terminar e ao fechar. O botão direito morre no vidro e na barra de controles, nunca no player de fora.

- **e o que ele desenha tocando, a casa cobre.** Há três marcas que nenhum

`playerVar` tira da tela, e cada uma tem uma resposta nossa: o **cartão de título** do topo (avatar, nome do vídeo, canal) é segurado pela `.video-mascara` ; o **botão de link** e a **marca "YouTube"** das esquinas de baixo são fechados pelos pseudo-elementos `.video-controles::before/::after` — só as pontas, porque o meio da faixa é onde estes tutoriais trazem a legenda queimada; e a **bolha cinza de play/pause** do centro

é coberta pelo próprio selo da casa, que entra maior (`.video-vidro.pulso`) exatamente por cima dela. A janela da cobertura é `JANELA_DAS_MARCAS`, medida no player real: o cartão dura ~7 s e a bolha passa dos 5 s e some antes dos 9 — logo, 9 s. Ela cai no `PLAYING` (inclusive na retomada do meio) e sobe no `PAUSED`, quando o vidro opaco já assume o quadro inteiro. O único play/pause que aparece na tela é o da casa.

Migração e seed novas

`2026_10_24_000004_cria_a_galeria_de_videos` cria a tabela com índice único em `codigo` e índices em `(ativa, ordem)` e `categoria`. É aditiva e tem `down()`. O `CmsSeeder` insere os 24 tutoriais reais do site antigo só quando a tabela está vazia, então resemear não duplica nada.

---

13. Página de campanha: o funil que só existe para quem tem o link

A campanha de divulgação não é uma página do site. Ela nasce em **Admin → Comercial → Marketing (funil)**, recebe um endereço `/c/{slug}`, e é montada bloco a bloco no painel. Quem não tem o link não a encontra: ela não entra no menu, não entra no rodapé, não entra no `sitemap.xml` e nasce com `<meta name="robots" content="noindex,follow">` — a visibilidade, porém, é escolhida por campanha, então dá para virar uma delas pública para a busca sem mexer em código.

Cada campanha carrega a própria captura de lead, os próprios pixels e o próprio contador. É a página inteira do funil: a promessa, os benefícios, a oferta, o vídeo, a prova social, a urgência, o formulário, as dúvidas e o conteúdo de apoio — nove tipos de bloco, todos editáveis na tela.

O que uma página de campanha guarda

Coluna	O que guarda
`nome`	como a campanha aparece no painel
`slug`	o endereço `/c/{slug}`; trocar o nome antigo continua no ar
`caminhos`	histórico JSON de todo endereço já servido — é o que preserva o contador
`status`	`rascunho` (sem endereço), `publicada`, `pausada` (volta a 404)
`visibilidade`	`noindex,follow` (padrão), `index` ou `noindex,nofollow`
`pele`	`ouro`, `noite` ou `claro` — a mesma paleta de tokens do site, por folha
`titulo`, `meta_description`, `meta_keywords`, `og_imagem`, `canonical`	o que o buscador e o compartilhamento veem
`pixel_ga4`, `pixel_google_ads`, `conversao_google_ads`, `pixel_meta`, `pixel_tiktok`	os píxeis **da campanha**, não do site
`codigo_head`, `codigo_body`	o que o anunciante precisar colar a mais
`preco_cents`, `nota_media`, `num_avaliacoes`	a oferta e a nota do JSON-LD de `Product`
`nota_legal`	o aviso miúdo do rodapé do funil

Os blocos vivem em `funnel_blocks`, com `tipo`, `ordem`, `ativa` e um conjunto fixo de campos de texto (`titulo`, `texto`, `corpo`, `itens`, `imagem`, `cta_*`, `limite_em`). **O tipo decide quais campos aparecem**: um bloco de vídeo não mostra o campo de itens, e um bloco que muda de tipo perde o que o novo tipo não usa — não há texto velho escondido em outro lugar da página. A lista de tipos mora em `config('golden.funil_blocos')`, com os rótulos, ícones, descrição, dica de preenchimento e as colunas de cada item; criar um tipo novo é acrescentar um bloco na config, sem controller e sem view.

O `itens` de um bloco é texto digitável, não tabela: **uma linha por item**, **colunas separadas por** `|` e, nas colunas de lista, itens separados por `;`. A barra do lado do campo mostra a ordem das colunas que aquele tipo espera.

No bloco **Texto sobre a foto** a imagem é plano de fundo da faixa inteira, não quadro ao lado do texto: ela cobre a seção de borda a borda e escurece por baixo do texto com o mesmo véu em gradiente já aprovado no herói do site. No celular o véu fica uniforme, porque a coluna é estreita demais para um degradê lateral. Por isso o campo se chama **Foto de fundo** e a dica pede paisagem (1600 × 900 px). Sem foto, o bloco é texto sobre a pele da página.

O vídeo da campanha toca na própria página

O bloco **Vídeo de vendas** é o `@include('site.partials.video-player', …)` da casa — a mesma peça do modal da galeria (seção 12), sem modal e sem link de saída. O bloco não abre janela nova nem leva ninguém para o YouTube: ele toca na faixa. Ao clicar em assistir, a API do YouTube entra em cena (só então — até ali a página não tem um byte de terceiros no caminho crítico) e monta o player sobre `.video-quadro`. O quadro entra em cena **coberto e calado**: o vidro da casa só levanta no primeiro frame tocando, e o play é sempre um toque em controle nosso. Sobre o quadro há um **vidro** que fica com o clique e com o botão direito — é o que devolve ao visitante um menu vazio em vez do menu de fora. Os únicos controles que existem são os da casa, embaixo do quadro: **tocar/pausar, mudo e a régua de volume**. O vídeo fica onde a faixa o pôs — não se solta da página ao rolar — e ao terminar volta ao cartaz e para no zero, pronto para assistir de novo com um toque.

Duas coisas valem conferir antes de mexer na folha:

- a entrada das seções (`main>section`, em `golden-site.css`) usa `backwards`, não

`both`: o estado final do keyframe já é o padrão do elemento, e um `transform` preso no fim faria da seção o bloco de contenção de qualquer `position: fixed` que morasse nela;

- o player entra no `.video-quadro`, nunca no `.video-yt`: a API troca o nó pelo

iframe e não repassa a classe dele.

Os carrosséis da campanha

Benefícios e **Prova social** são a mesma fileira do site vestido de card: `.carrossel` + `.swiper.js-carrossel`, com o comportamento em um lugar só (`initCarrosseis`, em `public/js/golden-site.js`). O que muda por faixa são os *breakpoints* e o ritmo, declarados no `data-passos` da view:

Faixa	Celular	Tablet (≥640/760)	Notebook (≥1100)	Avanço
Benefícios	1 por vez	2 por vez	**4 por vez, linha única**	4,2 s, **sentido reverso**
Prova social	1 por vez	2 por vez	até 3 por vez	5,5 s

- **sentido reverso** é o `data-sentido="reverso"` do bloco; o **pause ao segurar** no

celular é o `touchstart`/`touchend` do carrossel compartilhado, e **ao passar o mouse** é o `pauseOnMouseEnter` — os dois já vinham do site;

- quando tudo cabe na tela, a fileira está **travada** (`isLocked`): ela para de andar

e a fileira de controles some (`.carrossel.travado .carro-ctl{display:none}`), para não oferecer navegação a quem não tem para onde ir;

- **não há** `loop`, e sim `rewind`: com poucos slides o `loop` duplica o miolo e

reclama no console; o `rewind` volta ao início sem inventar item;

- na tela larga (≥721 px) os títulos, os resumos, a contagem regressiva e as chamadas

de **Benefícios**, **Prova social** e **Urgência** ficam **centralizados**; no celular seguem alinhados à esquerda.

A prova social não depende do que alguém digitar no bloco: `FunnelBlock::depoimentos()` põe na frente os **depoimentos aprovados da plataforma** (nome, cidade/papel, nota, foto quando há e o selo do Google para os importados) e soma, sem repetir ninguém, os itens digitados no próprio bloco. O nome decide a duplicidade — inclusive quando a mesma pessoa foi aprovada duas vezes. Para a faixa ficar cheia, **aprove ao menos cinco depoimentos** em *Admin → Conteúdo → Depoimentos* (ou importe os do Google com `php artisan depoimentos:google`, que pede a `GOOGLE_PLACES_API_KEY` no `.env`); o sistema não inventa avaliação.

O Swiper (154 KB de JS) é da casa inteira, mas **só é chamado pela campanha que tem faixa deslizante**: `FunnelPage::temCarrossel()` decide, na view, se a folha e o script entram nos `@stack` da carcaça. Campanha sem carrossel não paga por ele.

A tecla de ar da campanha

Na listagem e no contador de cada campanha há um botão de ligar e desligar. Ele não inventa um status novo: escreve o mesmo `publicada` / `pausada` do formulário, só que sem reescrever a página inteira. Confirmado em SweetAlert2, respondido em toastr, e o ato fica na auditoria (`funil_publicada`, `funil_pausada`).

O que a tecla garante:

- **campanha sem bloco nenhum não sobe.** A tecla avisa e manda montar pelo menos

um bloco antes — senão ela abriria um endereço em branco para quem clicou no anúncio;

- **pausar devolve 404**, como sempre devolveu, e **não zera o contador**: a página

guarda todo endereço que já teve (`caminhos`), então o tráfego que ainda chega pelo link antigo continua sendo registrado e volta a aparecer quando ela reabre;

- o menu do painel chama a tela de **Marketing (funil)**, que é o nome que o

operador usa para a coisa.

SEO e compartilhamento da campanha

A página monta `<title>`, descrição, palavras-chave, `robots`, `canonical`, `hreflang`, todo o bloco Open Graph e Twitter, e o JSON-LD do site: `LocalBusiness` sempre, `FAQPage` quando há bloco de dúvidas preenchido e `Product` com `Offer` e `aggregateRating` **só quando o preço estiver preenchido** — sem preço na tela não há oferta a declarar ao Google, e o seletor não inventa valor. Apagar o preço no painel derruba o `Product` na hora.

A carcaça é `resources/views/layouts/funil.blade.php`: leva a folha do site (`golden-theme` + `golden-site`) mais `golden-funil.css`, que só define as três peles. Não há navegação para o resto do site no topo nem no rodapé — quem chegou por um anúncio tem um único caminho a seguir, que é a chamada da página. O rodapé guarda a nota legal da campanha e o link da política de privacidade.

O clique, o cadastro e a conta

O funil não reimplementa o rastreio: ele usa a régua que já existia (`Atribuidor`, `Rastreio`, `config('golden.ids_clique')`). Toda visita a `/c/...` registra a origem do clique (`gclid`, `fbclid`, `ttclid`, `msclkid`, `li_fat_id`, `yclid`, `igsh`, `epik` e os demais) no cookie de visitante, e o cadastro herda essa jornada:

- o lead guarda `funnel_page_id`, `clique_id` e `clique_valor` — o nome do

parâmetro é a prova de qual anúncio gerou o cadastro;

- a página do lead vem do campo oculto `funnel_page_id` do formulário, e, quando

ele falta, do endereço em que a visita começou (`landing_path`);

- **rascunho e página pausada não recebem cadastro**: o painel só aceita id de

página publicada, senão qualquer POST atribuiria lead a uma campanha guardada;

- o pixel de conversão do Google Ads e os píxeis de Meta/TikTok disparam no

envio do formulário, medindo **só aquela campanha**.

A tela `Admin → Comercial → Marketing (funil) → nome da campanha` é o contador: visitas do período (com únicos e o total desde sempre), telas vistas, cliques medidos, cadastros, a taxa visita→cadastro, vendas fechadas, a fatia de tráfego pago e os robôs contados à parte. Abaixo, o gráfico dia a dia com visitas e cadastros na mesma escala, as três tabelas de origem (canal, rede, dispositivo), o **clique por clique** de cada anúncio, a lista paginada de visita por visita com a jornada de cada uma, e os últimos cadastros da campanha — com link para a listagem de leads já filtrada por ela.

Trocar o `slug` não zera nada: o endereço antigo entra no histórico `caminhos` antes de a mudança ir ao banco, continua servindo a mesma página e continua somando visitas no contador.

Cache e leveza

Assim como o resto do CMS, a campanha inteira (página + blocos ativos) sai do cache `golden.paginas_funil`: `/c/{slug}` com o cache aquecido não faz nenhum SELECT, e a primeira leitura do dia custa as mesmas poucas consultas das outras páginas públicas — o teste `LevezaDasPaginasTest` inclui o endereço da campanha semeada. Gravar página ou bloco no painel derruba a chave na hora.

Campanha pronta no primeiro acesso

O `FunnelSeeder` cria a campanha **Campanha de frota — J16 4G/2G** (`/c/frota-j16`) com os nove blocos na ordem de leitura, usando os kits e as dúvidas reais do `config('golden.php')` e as fotos que já vêm no projeto. Ele não inventa preço nem píxel: o operador preenche os dois no painel quando a campanha for ao ar, e os pacotes de preço ficam em branco para a equipe comercial confirmar. Roda uma única vez — se já houver blocos, o seletor sai sem duplicar.

Migrações novas

- `2026_10_24_000005_criam_as_paginas_de_funil` cria `funnel_pages` e

`funnel_blocks` (índices em `slug`, `status` e `(funnel_page_id, ativa, ordem)`);

- `2026_10_24_000006_amarram_o_lead_a_pagina_de_funil` acrescenta em `leads` as

colunas `funnel_page_id`, `clique_id` e `clique_valor`.

Ambas são aditivas e têm `down()`. Como todo migration deste projeto, elas precisam ser conferidas no **MySQL da hospedagem** antes de dadas por prontas: o SQLite dos testes não reclama de comprimento de `varchar`.

13.1 O assistente que atende o público

O sistema tem um robô de atendimento com rosto, nome e memória próprios. Ele atende **dentro do painel** (três telas no grupo **Atendimento**) e **fora dele**, na bolha que sobe no canto do site e de cada página de campanha. Nada dele se configura por arquivo: o que existe é ficha no banco, chave no cofre do painel e interruptor na tela.

As três telas

- **Assistente virtual** (`/admin/assistente`) — a persona: nome, cargo, saudação, tom de fala,

regras escritas pelo administrador, o rosto (arrastar e soltar, 96 × 96 px, até 2 MB) e a voz das respostas em áudio. É desta ficha que se monta a ordem que o provedor recebe; as regras entram na ordem exata em que foram escritas, sem reescrita. Quatro interruptores dizem onde ele atende: ligado, no site, na página de funil, no chat do chamado. O cartão **Bolha de atendimento**, na mesma ficha, edita o que o visitante lê no botão flutuante: a dica ao passar o mouse (aceita `:nome`), o ícone de quem não tem rosto, o selo de status, o texto do campo de mensagem e o link para falar com uma pessoa — em branco, vale o padrão de `config/assistente.php`.

- **O que o assistente sabe** (`/admin/assistente/conhecimento`) — a base de respostas. Um assunto

por linha, com resposta, tags, origem e endereço da fonte. A busca é léxica e ponderada (título pesa 3, assuntos 2, corpo 1) sobre o piso `assistente.resposta.minimo_de_fonte`; abaixo do piso a máquina **não responde**, ela chama a equipe. Sem índice vetorial de propósito: na hospedagem compartilhada um atendimento que depende dele é um atendimento que para.

- **Conversas do site** (`/admin/assistente/conversas`) — a fila dos visitantes anônimos, o

equivalente do lado público da fila de chamados. Aberta, aguardando a equipe (âmbar no menu) ou encerrada. Responder aqui assume o fio: a fala sai assinada pelo login da equipe e a máquina sai de cena no resto da conversa — o cliente nunca recebe duas respostas para a mesma pergunta.

Ligar a inteligência

Em **Sistema** → **Configurações** → **Chaves de integração** entram, cifradas no banco, as duas chaves que existem hoje: `ASSISTENTE_GEMINI_KEY` e `ASSISTENTE_OPENAI_KEY`. O `.env` é só plano de fundo para quando o banco está vazio; gravado pelo painel, manda o banco. Cada capacidade (conversa, visão, documento, transcrição, voz) escolhe provedor e modelo na mesma tela.

Um provedor sem chave **não é erro**: a capacidade fica desligada, a ficha do assistente mostra a pergunta *"ele consegue responder?"* dizendo qual capacidade está manca, e o atendimento continua pelas mãos da equipe. O código nunca sai do sistema por falta de credencial.

A bolha no site e no funil

O botão flutuante do assistente ocupa o andar de cima da calha de flutuantes: ele fica logo acima do botão do WhatsApp, que continua no pé da tela, e o balão abre a partir dele. O botão flutuante, o balão e a conversa anônima vêm com a página, e a folha (`public/css/golden-assistente.css`) e o script (`public/js/golden-assistente.js`) **só entram no HTML quando o interruptor do canal está ligado** — desligado, não sobra um byte de robô na página do visitante. As três rotas JSON são `POST /assistente/abrir`, `POST /assistente/fala` e `GET /assistente/falas`, com throttle de 30, 20 e 120 por minuto e por visitante.

Como se comporta:

- uma conversa por navegador (na sessão), e a saudação é escrita **uma única vez**;
- com o balão aberto a varredura é de 5 s; fechado, só as conversas passadas para a equipe são

varridas, a cada 30 s, e nunca com a aba em segundo plano;

- **robô de busca não vê a bolha e não abre conversa** — o índice do Google continua recebendo o

site, não uma fila de perguntas fictícias;

- sem fonte confiável, o visitante lê a frase de escalada e a equipe recebe a conversa por e-mail

(`contato_email`); nada de preço, prazo ou cobertura inventado;

- havendo formulário na página, o botão "falar com uma pessoa" rola até ele; não havendo, leva ao

`/contato` da casa com o resumo da conversa já escrito no campo de mensagem — um formulário por assunto, o resto são links;

- o cadastro feito depois da conversa **amarra as duas**: a ficha do lead traz o resumo do que o

visitante já contou, com canal e página de origem registrados.

A ficha é lida em cache (`golden.assistente_bolha`), como o rodapé e o menu, porque **nenhuma página pública emite** `SELECT` para montar o que está no ar; `CachesContent` derruba a chave no instante em que o painel grava, então desligar o robô ou trocar o rosto vale na hora.

Migração nova

`2026_10_26_000001_creiam_as_tabelas_do_assistente` cria `assistente`, `conhecimento_itens`, `conversas`, `conversas_mensagens` e `assistente_api_logs`. É aditiva e tem `down()`. Foi conferida no **MySQL da hospedagem** (646 ms, `DONE`) antes desta entrega — o SQLite dos testes não reclama de comprimento de `varchar`, e é assim que um migration escapa.

O que ainda não está no ar

A arquitetura está pronta e testada; estas partes ainda esperam:

- responder dentro do **chat do chamado** (o interruptor `no_chamado` nasce desligado);
- **áudio, imagem e documento** no chat — transcrever o que chega, ler o que é imagem, responder na

voz escolhida — que dependem de chave de visão/transcrição/voz;

- **aprender da internet** pelo rastejador de domínios permitidos (as páginas lidas chegariam em

`conhecimento_itens` com origem, endereço e data);

- **criar campanha no Google Ads** por pedido direto do administrador.

Para ver localmente: `php artisan serve --host=127.0.0.1 --port=8099`, abrir `/` ou `/c/frota-j16` com um navegador comum, e `/admin/assistente/conversas` com `admin@golden.com.br` / `golden123`.

---

14. Guia de uso dentro do painel

O painel documenta a si mesmo. **Guia de uso** é a última linha da barra lateral — item solto, fora dos grupos, porque é consulta e não área do sistema — e abre `/admin/guia` **em outra aba** para qualquer papel que entra no painel: vendedor, visualizador e administrador leem o mesmo texto. É a única tela do painel que se abre assim, e de propósito: consultar não pode custar o formulário que se estava preenchendo. Não há pacote extra, tabela nova nem comando de instalação; a rota está no grupo `admin` com `auth`, e todo o conteúdo vem de um arquivo de configuração.

O que tem dentro

São 30 seções em 7 grupos, na ordem em que a pessoa encontra a tela: **Começando** (o que é o painel, a rotina do dia), **Comercial** (leads, clientes, campanhas, atribuição, Google Ads, criativo, funil), **Vendas** (loja, pedidos, catálogo), **Atendimento** (chamados, contratos e as três telas do assistente), **Conteúdo do site** (páginas, imagens e textos, depoimentos, vídeos, perguntas frequentes), **Portal do cliente** e **Sistema** (notificações, configurações, crons, usuários, auditoria, ambiente, diagnóstico).

Cada seção tem resumo, blocos e — quando documenta uma tela — o botão **Abrir a tela**, que só aparece para rota registrada: guia apontando para rota renomeada é guia desmentido pelo sistema. A coluna da esquerda é o índice, presa à tela como uma barra lateral; a da direita mostra **uma seção por vez**, a que foi clicada. O clique troca o cartão da leitura e **não rola a página**: o corpo do cartão tem rolagem própria, e o endereço ganha `#secao` por `history.replaceState` — trocar o endereço sem pedir ao navegador que role até a âncora. O `#secao` no endereço continua valendo, e é ele que o link **Como usar esta tela**, no alto de cada tela, usa para abrir já na seção certa.

A busca ignora acento ("contrato" acha "Contratos"), esconde o que não casa, some com o título do grupo que ficou vazio e abre a primeira seção que casa; sem resultado, o índice avisa e a leitura continua onde estava. No celular, onde não existe "à direita", o índice ganha altura própria e rola por dentro, e a seção aberta é trazida para a tela depois do clique.

Seis tipos de bloco

`config/guia_painel.php` é a fonte única, lida por `App\Support\GuiaPainel`. A tela do guia, o cartão dobrável do Google Ads e a linha do cabeçalho consomem os mesmos blocos, pelo mesmo partial — o que está escrito no guia é literalmente o que a tela diz, e corrigir um lugar corrige todos.

Bloco	Para quê
`texto`	um parágrafo; aceita `**negrito**` e `` `código` `` no meio da frase
`passos`	sequência de linhas com ícone, título e explicação — **sem número**
`lista`	itens soltos
`codigo`	comando, linha de `.env`, cron: monoespaçado, rolagem própria, copiável sem quebra
`tabela`	cabeçalho e linhas
`aviso`	a frase marcada à esquerda, com `tom` (`gold`, `teal`, `blue`, `violet`, `green`, `amber`, `red`, `neutral`)

Nada aqui recebe numeração. Um passo é ícone + título + texto; a ordem já está na lista, e um "1." desenhado sobre ela é enfeite, não orientação — a regra que vale para as páginas do site vale para a documentação, e um teste não deixa voltar. Os tons de aviso vêm dos tokens do tema, cada claro com seu par no escuro.

A linha discreta em cada tela

No cabeçalho de toda tela que tem seção aparece uma linha discreta — **Como usar esta tela** — que leva à âncora certa (`admin.leads.*` → `#leads`, `admin.google-ads.*` → `#google-ads`). As ~20 telas do painel estão cobertas por um único `@include('admin.partials.ajuda-rota')` no layout, sem editar uma linha de cada view. Quem decide a seção é o mapa `rotas` do mesmo arquivo de configuração, e o prefixo mais longo vence quando mais de um cabe: `admin.google-ads.criativo` tem seção própria, `admin.google-ads.index` cai na do Google Ads. Rota sem seção não oferece link — a linha não promete documento que não foi escrito.

Escrever no guia

Mexer na documentação é mexer num array PHP, sem tocar em view:

```
$EDITOR config/guia_painel.php          # a fonte única do texto
php artisan test --filter=GuiaDoPainelTest   # a régua inteira do guia
```

Uma seção nova precisa de `id` (a âncora usada nos links), `grupo`, `titulo`, `icone` (FontAwesome, os mesmos da barra lateral), `resumo`, `blocos` e, se documenta uma tela, `rota`. Os testes reclamam de:

bloco sem o que desenhar, rota ou prefixo que não existe, número voltando no título de um passo, tom de aviso sem par na folha, item do menu sem seção e texto que sobra em inglês ou sai com marca crua (`**`, crase).

---

15. Central de crons: a agenda do painel, sem tocar no cPanel

A hospedagem tem **uma** linha de cron registrada, e ela não muda: chama `php artisan schedule:run` a cada minuto (seção 9). O que roda depois dela não está escrito no código — está na tabela `cron_tarefas` e se edita em **Sistema** → **Crons**, tela de administrador. Incluir, editar, pausar, excluir e **Rodar agora** são gestos do painel; a crontab do servidor não é editada de novo durante a vida da instalação. Na barra lateral a linha carrega uma bolha vermelha com o número de tarefas em falha.

As quatro tarefas que a instalação semeia

Migrar sem semear deixaria a fila de e-mails parada em silêncio, então a migration já escreve as tarefas que antes estavam fixas em `bootstrap/app.php` — mais a poda das notificações, que é nova:

Tarefa	Frequência	Comando
Drenar a fila de envios	a cada minuto	`queue:work --stop-when-empty --max-time=50`
Sincronizar o Google Ads	a cada quinze minutos	`google-ads:sync --agendamento`
Podar o registro de tráfego	toda terça, às 03:30	`rastreio:podar`
Podar as notificações lidas	de hora em hora, no minuto 15	`notificacoes:apodrecer`

As quatro nascem marcadas como **sistema**: podem ser pausadas e ter a frequência trocada, mas não apagadas. Apagar o worker da fila é o jeito certo de perder todo e-mail do portal sem que nada quebre, nada reclame e ninguém receba. Tarefa criada pela equipe, essa sim, se exclui.

A frequência não é escrita à mão. A tela oferece presets — de minuto a minuto até mensal, passando por semanal com dia da semana — e `CronTarefa::resolverExpressao()` monta a expressão cron a partir de preset + hora + dia, com minuto e hora zero-padded (`03:30` vira `30 03 * * *`, não `30 3`). A expressão fica guardada na coluna própria para a listagem ler sem calcular; quem manda continua sendo a preset. Tarefa diária sem hora preenchida assume `03:00`, porque tarefa de manutenção é de madrugada.

O que uma tarefa aceita como comando

O campo não é shell. `CronTarefa::erroDoComando()` fecha três portas, e a mensagem diz qual delas:

- **Lista negra** (`config/cron.php`): `tinker`, `migrate` e as variantes de rollback, `db:wipe`,

`queue:flush`, `key:generate`, `storage:link`, `app:define-senha`. Nenhum deles pertence a uma agenda automática, e cada um tem o motivo escrito na mensagem em vez de um "não permitido".

- **Sintaxe**: `;`, `&&`, pipe, redirecionamento, crase e quebra de linha estão fora. As flags

aceitam letras, números, travessão, dois-pontos, barra e igual.

- **O registro do `artisan`**: o nome do comando precisa existir em `Artisan::all()`. Comando

digitado errado devolve erro de digitação, não tarefa fantasma que nunca roda.

O select do formulário sugere o caderno de comandos do sistema (fila, Google Ads, podas, depoimentos do Google, caches), mas a validação é a mesma para o que for digitado.

Rodar agora existe para conferir sem esperar o próximo minuto, e recusa os comandos da fila (`queue:work`, `queue:listen`, `queue:restart`): o worker é o mecanismo que executa os jobs, e chamá-lo dentro de uma requisição prende a tela por cinquenta segundos para fazer o que o cron já faz sozinho. A recusa aparece em toastr com o motivo.

O que fica registrado

`cron_execucoes` é a caixa-preta: `inicio`, `fim`, `status`, `duracao_ms`, `saida`, `erro`, `acionada_por` (`agendamento` ou `painel`) e o `user_id` de quem apertou o botão. A ficha da tarefa mostra as vinte últimas execuções; a tarefa guarda apenas o último batimento denormalizado, para a listagem ler barato. A saída é cortada em 6.000 caracteres — um `queue:work` falando por cinquenta segundos cospe log de cada job, e guardar tudo apagaria o que importa.

Cada tarefa nasce com `sem_sobreposicao`: o agendador registra `withoutOverlapping()`, então a execução de agora não começa enquanto a anterior não devolve. Uma linha parada em `executando` depois que o processo morreu é fechada como falha no início da execução seguinte, com o motivo escrito — histórico com linha aberta para sempre é histórico que ninguém confia.

O batimento e o diagnóstico que ele separa

O topo da tela mostra se o agendador está vivo. Ele não mede tarefa nenhuma: mede a passagem do `schedule:run`, gravando um timestamp em cache a cada minuto (`golden.cron_batida`, tolerância de 120 segundos). Isso separa dois mundos que parecem iguais de fora:

- **"Sem batida há duas horas"** — a linha da crontab da hospedagem sumiu ou o PHP do caminho mudou.

Nenhuma tarefa da agenda roda, inclusive a fila de e-mails.

- **"Falhou"** — a agenda está viva e *esta* tarefa é que devolveu erro.

Além de responder ao vivo, a poda de notificações faz a ronda: tarefa ativa cujo último batimento passou da tolerância da própria preset (o dobro da janela, com folga) gera um aviso de **agenda sem batimento**, uma vez por dia por tarefa. É a única forma de saber que a agenda parou sem ninguém estar olhando; o próprio cron não pode se vigiar.

Migração nova

`2026_10_26_100001_creiam_a_central_de_crons` cria `cron_tarefas` e `cron_execucoes`, com índices por situação e por histórico e chave única no nome, e semeia as quatro tarefas de sistema. É aditiva, tem `down()` e não toca em nenhuma tabela existente.

Testes

`tests/Feature/CentralDeCronsTest.php` (15) cobre a expressão resolvida a partir do preset, a validação dos três casos de comando, o registro do histórico no "Rodar agora", a recusa dos comandos da fila, a proteção da tarefa de sistema, a tolerância de cada preset e o aviso de agenda sem batimento — além de a tela ser fechada para vendedor.

---

16. Central de notificações: o sino, a página e a ficha

Todo aviso que o sistema precisa contar à equipe — lead novo, pedido, chamado aberto, resposta do cliente, depoimento novo, conversa escalada, conta do portal, Google Ads desconectado, tarefa do cron que falhou, agenda sem batimento — nasce dentro do painel, e não por e-mail. A central tem três superfícies e uma regra que organiza todas: **uma linha por destinatário**. Cada pessoa tem as suas, com o próprio `lida_em`; abrir uma notificação a tira do sino de quem abriu, e de mais ninguém.

Notificações é um item solto da barra lateral, logo depois do Dashboard — o sino tem bolha própria e a página é de cada pessoa, então ela não entra num grupo, que no menu é área e não caixa de entrada. A rota está no grupo `admin` só com `auth`: vendedor, visualizador e administrador recebem e leem. A separação não é de middleware, é da linha: a ficha e a exclusão de notificação alheia devolvem 404.

As rotas

Rota	Para quê
`GET /admin/notificacoes`	a página dedicada: KPIs, filtros e histórico
`GET /admin/notificacoes/{id}`	a ficha de cada aviso — e o instante em que ele sai do sino
`POST /admin/notificacoes/lidas`	marcar todas como lidas
`DELETE /admin/notificacoes/{id}`	excluir da própria central
`GET /admin/notificacoes/sino`	o JSON do sino, `no-store`, `throttle:60,1`
`GET` e `POST /admin/notificacoes/preferencias`	o que cada pessoa desliga, e os prazos do administrador

O sino

A primeira pintura vem do servidor (`admin/partials/sino.blade.php`); dali em diante `public/js/admin-notificacoes.js` consulta o JSON e reconstrói a lista no **mesmo formato** que a partial desenhou (`Notificacao::paraSino()`), para a linha não mudar de cara depois da primeira atualização. O contador mostra o total e satura em `99+`; o menu mostra oito linhas e manda o resto para "Ver todas", porque sino com trinta itens deixa de ser sino.

O intervalo é duplo: 45 segundos com a aba visível, 5 minutos com a aba escondida. Painel aberto numa aba de fundo não tem por que bater na porta do servidor. Sem rede, o erro é engolido e o sino espera a próxima volta — nada de toastr por causa de um *polling*.

Ler é o gesto de resolver

A leitura é gravada na abertura da ficha, não num botão: o gesto de ler é o gesto de resolver, e um passo a mais transforma o sino numa caixa de entrada que ninguém esvazia. A ficha avisa em uma linha o que acabou de acontecer ("Lida agora. Esta notificação saiu do sino — o histórico da central continua aqui.") e não reescreve o `lida_em` de quem volta depois. **Marcar todas** esvazia o sino de uma vez sem apagar nada: o que foi lido continua listado, com a data da leitura.

A página tem KPIs (novas, lidas, recebidas hoje, vencendo em três dias), filtros por situação, tipo e busca no título e no texto, e a mesma paginação de vinte linhas do resto do painel. A ficha — `/admin/notificacoes/{id}` — é genérica: mostra a mensagem inteira, o tipo com o selo do tom, as datas de recebimento e leitura, o que aconteceu (os dados etiquetados que o aviso carrega) e o botão **Abrir o registro**, que leva ao chamado, ao pedido, ao lead ou à tarefa do cron de origem.

O catálogo

`config/notificacoes.php` é a fonte única: rótulo, ícone FontAwesome, tom, audiência, rota de destino e, quando o tipo envelhece rápido, prazo próprio. Um tipo novo de aviso entra por aqui, sem migration e sem view — a ficha já sabe montar qualquer tipo a partir do que a linha carrega.

Tipo	Quem recebe	Abre
`lead_novo`	equipe	ficha do lead
`pedido_novo`	equipe	ficha do pedido
`chamado_aberto`	equipe	chamado
`chamado_respondido`	equipe	chamado
`depoimento_novo`	administrador	fila de depoimentos
`conversa_escalada`	equipe	conversa do assistente
`conta_portal`	administrador	ficha do cliente
`ads_desconectado`	administrador	tela do Google Ads
`cron_falhou`	equipe	ficha da tarefa
`cron_pausado_demais`	administrador	central de crons

`App\Support\Notifica::disparar()` é o único ponto de disparo. Ele cria **uma linha por destinatário ativo**, respeita a preferência de cada um, aceita uma `chave` de deduplicação (única por pessoa e por chave, então o mesmo fato visto duas vezes não vira dois avisos) e devolve quantas linhas entraram **agora** — o que já estava lá não conta como aviso novo. Os pontos que antes só mandavam e-mail passaram a chamar esse serviço; o e-mail continua saindo exatamente de onde sempre saiu.

Preferências e prazos

Em **Preferências**, cada pessoa desliga o tipo que não quer ver no sino. A linha de preferência só existe para o tipo desligado — o padrão é receber —, e religar apaga a linha. Desligar silencia o sino de quem desligou, não a caixa de entrada: a central é o canal de dentro do painel e não substitui o e-mail.

O administrador define, na mesma tela, dois números globais gravados em `settings`:

- **Retenção** (padrão 30 dias): quanto tempo uma notificação **lida** fica guardada.
- **Expiração** (padrão 15 dias): quando uma notificação **não lida** vence e sai sozinha.

Os dois aceitam de 1 a 365 dias, valem para a próxima poda sem deploy e ficam registrados em *Auditoria* (`notificacoes_preferencias`). O `.env` só serve de plano de fundo quando o banco está vazio (`NOTIFICACOES_RETENCAO_DIAS`, `NOTIFICACOES_EXPIRACAO_DIAS`).

A poda

`php artisan notificacoes:apodrecer` faz três coisas, e é tarefa da agenda semeada de hora em hora:

- apaga o que foi lido além da retenção e o que venceu sem ser lido;
- apara o histórico da agenda em `cron.historico_por_tarefa` (500 linhas por tarefa — um cron de minuto escreve 1.440 por dia);

- avisa, uma vez por dia por tarefa, a tarefa ativa que passou da própria tolerância sem batimento.

Notificação que não morre vira sino ignorado: a pessoa aprende a fechar o que nunca vai ler.

Migração nova

`2026_10_26_100002_creiam_a_central_de_notificacoes` cria `notificacoes` (tipo, título, texto, rota e parâmetro, dados em JSON, destinatário, chave de deduplicação, `lida_em`, `expira_em`) e `notificacao_preferencias` (destinatário + tipo + `no_painel`, únicos por par). Índices por destinatário/situação e por vencimento; a FK do usuário cascateia na exclusão da conta. É aditiva e tem `down()`.

Testes

`tests/Feature/CentralDeNotificacoesTest.php` (16) cobre a linha por destinatário, a audiência de cada tipo, a deduplicação pela chave, o silêncio de quem desligou o tipo, a leitura que tira do sino sem afetar outra pessoa, o 404 em ficha e exclusão alheias, o JSON do sino (teto, `no-store`, formato), a expiração fora do sino, a rota de destino de todo tipo do catálogo, as preferências e os prazos globais.

---

17. Retorno e confirmação: uma camada só

Toda resposta do sistema ao usuário — o que deu certo, o que impediu a ação e a pergunta antes de um ato destrutivo — sai por um único lugar: `resources/views/partials/avisos.blade.php`, com a folha `public/css/golden-avisos.css`. Nada de caixa fixa no alto da página e nada de `alert()` ou `confirm()` do navegador: o primeiro quebra o layout em três linhas de português, o segundo é cinza do sistema operacional.

- **Feedback** (salvou, falhou, erros de validação) vira notificação *toastr* no

canto superior direito. O partial lê `success`, `status`, `info`, `warning`, `error` e o `$errors` da view, então qualquer controller que faça `->with('success', …)` ou `->withErrors(…)` já aparece na tela sem código novo.

- **Confirmação** de exclusão, cancelamento e atos parecidos usa **SweetAlert2**.

Basta escrever `data-confirmar="Excluir este usuário?"` no `<form>`; o componente `components/admin/delete.blade.php` já nasce com o atributo, e o interceptor do partial impede o envio até a resposta. Cancelar não navega.

- **Cinco superfícies, uma folha**: painel, portal do cliente, as telas de acesso

(`/login`, `/portal/entrar`, `/portal/cadastro`) e o site público incluem o mesmo partial. No site ele entra com `['confirmacoes' => false]`, porque nenhuma tela pública tem ato destrutivo — assim o SweetAlert2 não é nem baixado.

- **Tema**: a camada de aviso não cria cor própria. Ela consome os tokens de

`golden-theme.css` (`--login-surface`, `--t-*-fg`, e os dois novos `--aviso-sombra` e `--aviso-veu`), então o *toastr* escuro/claro acompanha o botão de tema. Por isso o site público passou a carregar `golden-theme.css` antes de `golden-site.css`.

- **O ícone do toastr**: a folha do CDN pinta um *sprite* de fundo em cada

notificação, e a regra dele usa `!important` com a mesma especificidade da nossa. A trava (`#toast-container>.toast{background-image:none!important}`) mora num `<style>` do próprio partial, logo depois do `<link>` do CDN, porque num empate de especificidade vence quem chega por último ao documento — assim um navegador com a stylesheet antiga em cache não reabre o mosaico de ícones. O desenho do tom (borda + glifo Font Awesome) continua em `golden-avisos.css`, que não declara `background-image` nenhum.

- **Português**: o Laravel 13 não traz texto de validação no repositório, então a

chave crua (`validation.required`) aparecia na tela em vez da mensagem. As traduções estão em `lang/pt_BR/` (`validation`, `auth`, `pagination`, `passwords`, com o mapa `attributes` que dá nome brasileiro a cada campo) e o instalador escreve `APP_LOCALE=pt_BR`. Não há nada extra a fazer na instalação; se você trocar o idioma, troque também `APP_LOCALE` e crie a pasta correspondente.

- **Travado por teste**: `tests/Feature/RetornoEConfirmacaoTest.php` exige que

toda página carregue o partial, proíbe `alert(` / `confirm(` em Blade e no JS público, proíbe `{{ session(…}}` / `{{ $errors->… }}` pintando caixa, e confere que a folha de avisos só usa token existente nas duas paletas. `tests/Feature/TraducoesPtBrTest.php` varre as regras de validação usadas em `app/` e reclama se alguma não tem texto em português.

Instalou e ainda apareceu `validation.required`? É cache de view antigo: `php artisan optimize:clear`.

---

18. Paginação: um view por superfície

Toda lista paginada do sistema — as 12 do painel, as interações da ficha do CRM e os chamados e pedidos do portal do cliente — sai pelo mesmo `resources/views/pagination/painel.blade.php`, registrado uma única vez em `AppServiceProvider::boot()`:

```
Paginator::defaultView('pagination.painel');
Paginator::defaultSimpleView('pagination.painel');
```

Nenhuma view do painel escreve `->links()` com view própria, e nenhuma folha do framework é publicada no projeto. Isso importa porque **o padrão do Laravel é o view do Tailwind**, e ele dentro do Bootstrap produz três sintomas que parecem três bugs diferentes:

- os botões saem como caixa branca sem estilo — as classes utilitárias

(`px-4 py-2`, `inline-flex`) não correspondem a nada carregado;

- o contador vem em inglês ("Showing 1 to 20 of 24 results"), porque aquele texto

é do próprio view do framework, não de um arquivo de tradução;

- e um **chevron SVG do tamanho da tela** aparece no meio da página: o ícone não

tem largura definida, então cresce até estourar o container.

O nosso view entrega `pagination pagination-sm` (Bootstrap 5), os números das páginas, o resumo em português ("1–20 de 24") e `aria-label` em português em cada enlace. Ele serve o `paginate()` e o `simplePaginate()` — o bloco de números só existe quando o paginador sabe o total.

A cor também vem de uma fonte só. O Bootstrap pinta a página ativa com o ciano `#0dcaf0`, que não tem par no tema escuro; `public/css/admin-golden.css` redefine as variáveis `--bs-pagination-*` a partir dos tokens do painel (`--gold`, `--brand-gold-ink`, `--bs-border-color`, `--bs-tertiary-bg`), então claro e escuro acompanham o botão de tema sem regra extra. Como o portal inclui a mesma folha, a regra vale para os dois.

`PaginasDoPainelTest::test_a_paginacao_e_do_painel_e_nao_do_framework` abre duas listas e recusa qualquer rastro do view do framework. É o que impede a volta do problema se o provider mudar.

A superfície pública tem o view irmão (`resources/views/pagination/site.blade.php`): o site não veste Bootstrap nem Tailwind, então os números vêm com as classes e os tokens do `golden-site.css` — disco dourado na página atual, chevron de Font Awesome nas setas, seta desativada (não sumida) na primeira e na última página. Ele é chamado no lugar do `->links()` **uma única vez**, na galeria de vídeos, e não registrado como padrão: no `boot()` do provider ainda não se sabe por qual rota o pedido entrou, então não há como escolher a folha pela superfície. É também o motivo de a página atual carregar `class="pagina pagina-on"` e não só `pagina-on` — sem a classe base ela não recebe a caixa compartilhada e encolhe até virar um traço dourado em volta do número.

---

19. Backup e restauração

```
# banco
mysqldump --single-transaction -u USUARIO -p NOME_DO_BANCO | gzip > banco-$(date
+%F).sql.gz
# arquivos enviados
tar czf storage-$(date +%F).tar.gz storage/app/public
```

Restaurar:

```
gunzip < banco-2026-10-23.sql.gz | mysql -u USUARIO -p NOME_DO_BANCO
tar xzf storage-2026-10-23.tar.gz
php artisan storage:link
```

Guarde o `.env` fora do repositório (ele tem a `APP_KEY` e as credenciais): sem ele, os dados criptografados e as sessões não voltam.

---

20. Problemas comuns

Sintoma	Causa	Solução
Erro 500 com `Your Composer dependencies require a PHP version ">= 8.3.0"`	o PHP **da web** é antigo, mesmo com o `php -v` do SSH mostrando 8.3	MultiPHP Manager → 8.3; se o seletor da conta não chegar lá, `--php-web=ea-php83` / a linha `AddType` no `.htaccess` (seção 2)
Site mostra a lista de arquivos e pastas	docroot na raiz sem `.htaccess`, e `Options Indexes` ligado	rode o instalador (ou copie o `.htaccess` da seção 4.7)
`Route [admin.xxx] not defined` ao salvar	cache de rotas velho	`php artisan route:clear && php artisan route:cache`
Página quebra depois de editar um `.php` em `config/`	cache de configuração	`php artisan config:cache`
Lista paginada sai com caixa branca de "Anterior/Próximo", contador em inglês e um ícone SVG do tamanho da tela	o view de paginação do framework (Tailwind) voltou a ser o ativo	confira as duas linhas de `Paginator::defaultView()` no `AppServiceProvider` e rode `php artisan optimize:clear` (seção 18)
Na galeria pública, o número da página atual é um traço dourado apertado em volta do algarismo	o `<span>` da página atual perdeu a classe base `pagina` e não recebe a caixa compartilhada	`resources/views/pagination/site.blade.php` (seção 18)
Ícone sai como quadradinho (□) no lugar do símbolo, em qualquer tela	o CSS de ícone é servido pela casa e chama a fonte por caminho relativo: movido um dos dois, a fonte não é alcançada. A outra causa é o CSS servido por endereço diferente do arquivo, sem `Access-Control-Allow-Origin` na fonte	mantenha `public/css/fontawesome/css/all.min.css` ao lado de `public/css/fontawesome/webfonts/`, e chame o CSS só pelo `partials/icones.blade.php`. `php artisan test --filter=test_os_icones_sao_servidos_pela_casa_e_nao_por_um_cdn` percorre cada caminho declarado no CSS (seção 4.7)
Imagem enviada não aparece	`public/storage` inexistente	`php artisan storage:link`

Sintoma	Causa	Solução
Imagem enviada existe no servidor e a URL devolve **403**	o `.htaccess` da raiz bloqueia o caminho `/storage`	tire `storage` da regra de pastas de código-fonte (seção 4.7)
Upload falha com erro em `getimagesize` / `finfo`	`fileinfo` desligada no PHP da web	ative a extensão no seletor de PHP (seção 2)
Upload falha no meio com "passou do limite do servidor"	`post_max_size` menor que o campo	ajuste no painel de PHP (40 MB para vídeo + margem)
`could not find driver`	`pdo_mysql` desligado	ative a extensão no painel de PHP
`500` com `Permission denied` em `storage/`	permissões	`chmod -R 775 storage bootstrap/cache`
`Access denied for user`	usuário sem privilégio no banco	ligar usuário ao banco no painel MySQL
E-mail não sai	`MAIL_MAILER=log` ou credencial errada	conferir `storage/logs/email.log`; depois configurar SMTP
Aparece `validation.required` na tela em vez da mensagem	falta a tradução da framework, ou há cache de view velho	`php artisan optimize:clear`; confira `APP_LOCALE=pt_BR` e a pasta `lang/pt_BR/` (seção 17)
`550 ... account or domain may not exist` no `failed_jobs`	o destinatário é um endereço que não existe fora do exemplo (domínio sem MX, como `@exemplo.com.br`)	esperado em conta de demonstração com e-mail fictício; o aviso à equipe saiu, o que não existe é a caixa de exemplo. Troque o `email_portal` da ficha por um endereço real e o ciclo fecha
Falha de login no SMTP com senha certa no `.env`	`MAIL_PASSWORD` sem aspas e com `#`	escreva `MAIL_PASSWORD="..."` — sem aspas o `#` vira comentário
Página do site antiga depois de publicar	`opcache` do servidor	`php artisan optimize:clear`; em cPanel, reinicie o PHP
A senha do painel "não funciona" no servidor	a senha instalada é forte, não é `golden123`	leia em `~/backups-golden/acesso-painel.txt` ou defina outra com `php artisan app:define-senha`

Sintoma	Causa	Solução
O cliente não consegue entrar no `/portal`	o login do portal é o `email_portal`, não o e-mail da ficha do CRM	conferir o selo *conta ativa* na ficha (Admin → Clientes) e usar aquele e-mail; sem conta, ele cria em `/portal/cadastro`
O link de redefinir a senha morre no caminho	expira em 60 minutos e a assinatura depende da `APP_KEY`	reenviar por `/portal/esqueci-senha`; se a `APP_KEY` foi trocada, os links anteriores deixam de valer (é o esperado)
Contrato anexado não aparece no portal	o status está como `rascunho`	passar para assinado/vigente — rascunho é invisível para o cliente por projeto
O PDF do contrato abre 404 no portal	arquivo fora de `storage/contratos/`, ou o download tentou por URL direta	reanexar pelo dropzone e usar `/portal/contratos/{id}/arquivo`, que é a única porta de saída

Os logs ficam em `storage/logs/laravel.log`; no cPanel também existe `public/error_log`, que o `.htaccess` bloqueia para visitantes, e o erro real continua visível para você pelo painel ou SSH.

---

21. Segurança

- `.env` com `chmod 600` e fora do git; `APP_DEBUG=false` em produção.
- Senha do painel forte e única; sem `golden123` em servidor público.
- **Onde ela fica**: o instalador escreve login e senha em

`~/backups-golden/acesso-painel.txt` (permissão `600`, fora do docroot e do git). A tela de `/login` só mostra a dica de demonstração com `APP_ENV=local` — em produção a tela não entrega credencial nenhuma, e é assim que tem de ser.

- **Perdeu a senha**: pela SSH, sem precisar do painel,

`php artisan app:define-senha admin@golden.com.br 'nova-senha'` (a senha também pode ser digitada no prompt, sem eco). Use `--reativar` se a conta foi desligada. O comando lista os e-mails cadastrados quando o e-mail está errado e grava o evento na trilha de auditoria.

- Repositório público: nenhuma credencial, chave SSH ou cópia de banco entra nele.
- O `.htaccess` bloqueia código, segredos e logs; a entrega real é sempre pela

pasta `public/`.

- Arquivo enviado vira nome legível + sufixo aleatório, na pasta do próprio

campo; a extensão é a do conteúdo, então um script batizado de `.jpg` é recusado em vez de virar `.php` executável.

- O destino do envio (`campo`) decide a pasta; o navegador nunca escolhe caminho

no disco, e o descarte só apaga arquivo que comece na pasta do próprio campo.

- Backups de banco ficam fora do docroot, com `600`.
- Portal do cliente: sessão em chave própria (`login_cliente`), recurso de outro

cliente devolve **404** e não 403, `password` e `remember_token` estão em `$hidden` da ficha, o documento é só leitura no portal (é a chave do vínculo com pedido e contrato), e o cadastro tem honeypot, throttle por rota e consentimento LGPD obrigatório.

- O PDF do contrato não tem URL pública: sai só por `/portal/contratos/{id}/arquivo`,

depois de conferir o dono e que não é rascunho. O caminho gravado no formulário é revalidado contra a pasta do campo de envio.

---

22. Checklist depois de instalar

- [ ] `https://seudominio.com.br` abre e o menu funciona
- [ ] PHP da web conferido (`_chk.php` temporário, ou o aviso final do instalador)
- [ ] `/loja` lista produtos; página de produto mostra imagem e vídeo
- [ ] Checkout registra pedido e ele aparece no painel
- [ ] `/admin` abre com o login criado na instalação
- [ ] Campo de envio: arrastar imagem, vídeo e documento mostra preview e progresso
- [ ] Arquivo enviado pelo painel abre pela URL de `/storage/...` (200, não 403)
- [ ] Teste de barreira: `.env`, `app/` e `storage/logs/laravel.log` respondem 403
- [ ] Configurações do painel com WhatsApp, telefone, e-mail e endereço corretos
- [ ] *Conteúdo → Imagens do site* lista as 19 fotos, e *Conteúdo → Textos das páginas* lista

as sete páginas — trocar uma foto e salvar muda a página pública na hora

- [ ] SMTP preenchido (ou `log`, assumido de propósito)
- [ ] Retorno em português: errar a senha em `/login` deve abrir uma notificação

toastr no canto, nunca `validation.required` e nunca caixa dentro da página

- [ ] Confirmação: excluir um usuário no painel abre o diálogo SweetAlert2, e

Voltar não exclui nem recarrega a lista

- [ ] Paginação: uma lista com mais de 20 registros (Leads, Vídeos) mostra "1–20 de 45" e os

números em dourado — nunca caixa branca, nunca "Showing … results", nunca ícone gigante (seção 18)

- [ ] Paginação pública: `/videos` abre com 12 cartões, o resumo "1–12 de 24 tutoriais" e o

pager em disco dourado; `?page=99` volta para a última página e o filtro de categoria continua no enlace (seções 12 e 18)

- [ ] Entrega estática: `curl -sI https://seudominio.com.br/css/golden-site.css` responde `cache-control: public, max-age=31536000`, e o HTML responde `no-cache` (seção 4.7)
- [ ] Ícones: nenhuma requisição a `cdnjs.cloudflare.com/ajax/libs/font-awesome` no carregamento da página, e nenhum quadradinho (□) no lugar de símbolo (seção 4.7)
- [ ] Tecla de ar: em *Comercial → Marketing (funil)*, pausar a campanha deve devolver 404 no `/c/...` sem zerar o contador, e uma campanha sem bloco não sobe (seção 13)
- [ ] Cron do agendador: `php artisan schedule:run` a cada minuto na crontab da hospedagem (seção 9) e **Sistema** → **Crons** mostrando "Agendador vivo" — as quatro tarefas semeadas ativas, com histórico aparecendo depois da primeira batida (seção 15)
- [ ] Notificações: o sino no topo conta os avisos da pessoa, a página *Notificações* filtra por situação e tipo, e abrir uma ficha a retira do sino sem apagar o histórico (seção 16)
- [ ] Fila: abrir um chamado no portal e confirmar que a tabela `jobs` volta a zero no minuto seguinte — com `MAIL_MAILER=log`, o texto aparece em `storage/logs/email.log`
- [ ] Fila com erro: se `failed_jobs` tiver linha, ler o motivo nela, corrigir e rodar `php artisan queue:retry all`
- [ ] Tráfego: abrir `/admin/atribuicao` depois de algumas visitas reais e conferir se as origens aparecem
- [ ] Google Ads (opcional, seção 9.1): com as variáveis no `.env`, *Comercial → Google Ads* mostra o botão **Conectar conta Google Ads**; conectar, ver a tabela preencher sozinha e conferir em `/admin/google-ads/criativo` que um título de 31 caracteres fica vermelho
- [ ] Google Ads sem credencial: a mesma tela deve listar **qual** variável falta em vez de quebrar, e nenhum token de desenvolvedor aparece no HTML nem no log
- [ ] Guia de uso: **Guia de uso** na barra lateral abre em outra aba, o índice à esquerda abre uma seção por vez à direita sem rolar a página, a busca acha "contrato" sem acento, e cada tela com seção mostra a linha **Como usar esta tela** caindo na seção certa (seção 14)
- [ ] Campanha: `/admin/funil` mostra a campanha semeada; abrir `/c/frota-j16?ttclid=TESTE123` no navegador, cadastrar-se pelo formulário da página e conferir o clique, a origem e o cadastro na tela da campanha (seção 13)
- [ ] Vídeo: na campanha e na galeria, abrir o quadro **não** põe nada para tocar — ele nasce coberto pelo vidro da casa, sem autoplay e sem link de saída; só o toque em play (da faixa, do modal ou do próprio vidro) sobe o vídeo, junto com play/pause, mudo e volume da casa e sem menu de botão direito. Ao pausar ou terminar, o vidro desce de novo e o quadro da faixa volta ao cartaz (seções 12 e 13)

- [ ] Carrosséis da campanha: no notebook os benefícios ficam em **linha única** e a prova

social em até três por vez, com os títulos centralizados; no celular um por vez, arrastando para qualquer lado, parado ao segurar (seção 13)

- [ ] Depoimentos: aprovar **pelo menos cinco** em *Conteúdo → Depoimentos* para a faixa de

prova social encher; sem `GOOGLE_PLACES_API_KEY` no `.env`, o importador do Google não roda — e o sistema não inventa avaliação (seção 13)

- [ ] Portal: criar conta em `/portal/cadastro` sobre uma ficha que já existe no CRM e conferir que ela herda os pedidos de convidado
- [ ] Portal: entrar com o `email_portal` (não o e-mail da ficha), abrir um chamado e respondê-lo do painel — a conversa tem de ser a mesma nas duas telas
- [ ] Portal: anexar o PDF de um contrato assinado e baixá-lo pelo portal; com status `rascunho` ele não pode aparecer na lista
- [ ] Backup automático de banco e `storage/app/public`

Documento gerado de **docs/INSTALACAO.md** — a fonte é a mesma para este PDF e para o README do repositório. Nenhuma credencial aparece aqui: senha de banco, de painel e **APP_KEY** ficam apenas no **.env** do servidor.